

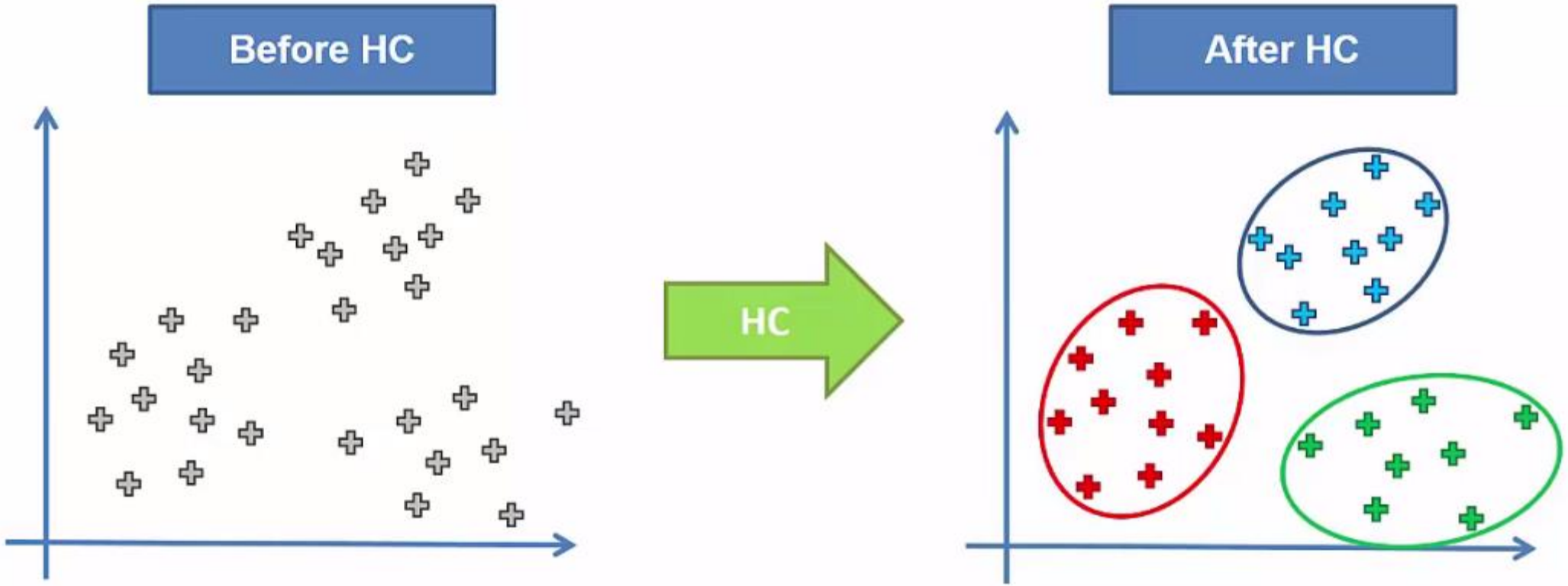
Hierarchical Clustering

In Machine Learning

Hierarchical Clustering

Find successive clusters using previously established clusters

What HC does for you



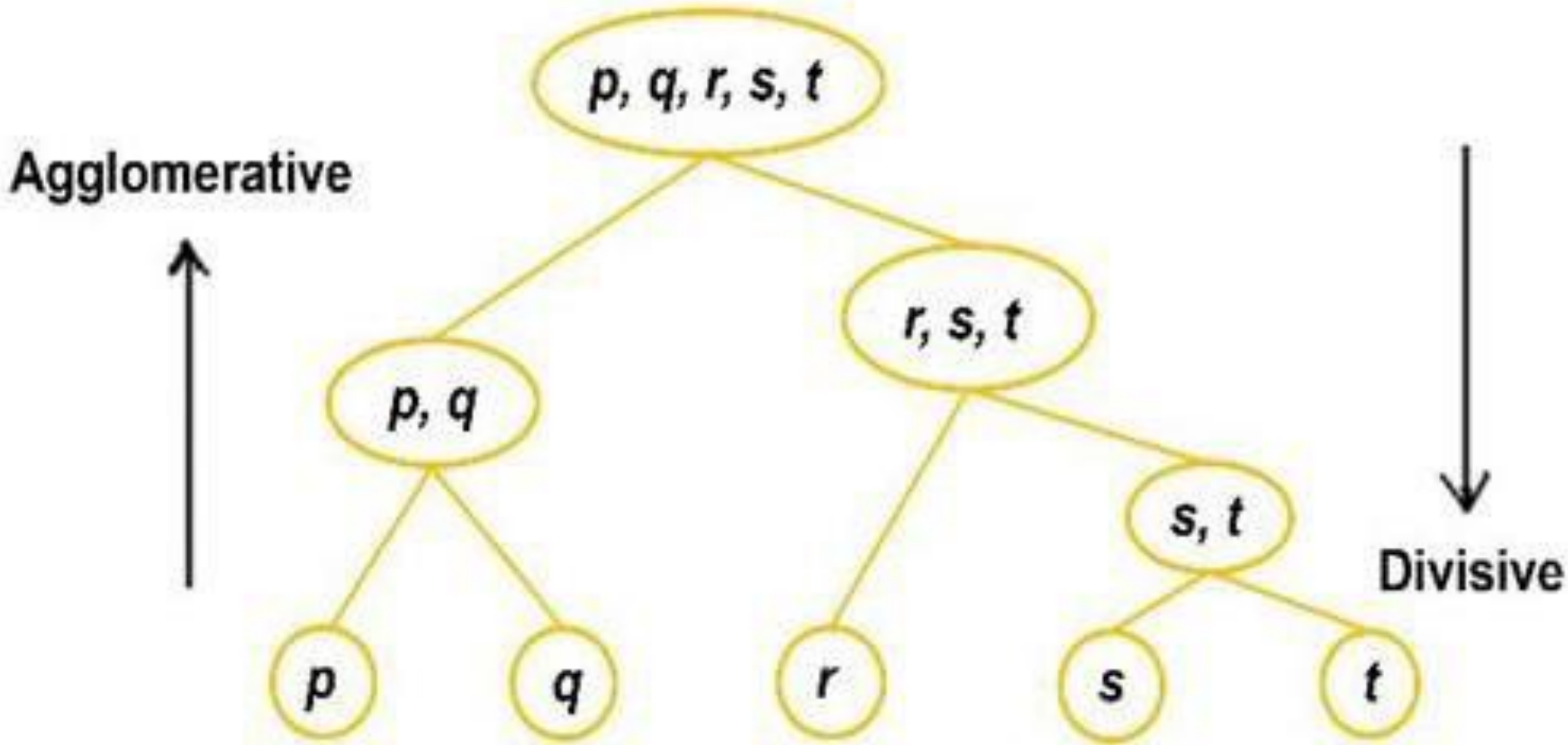
Same as K-Means but different process

Hierarchical Clustering

- Two types
  - Agglomerative (bottom-up)
  - Divisive (top-down)

- Agglomerative Algorithm
  - Begin with each data element as a separate cluster and merge them into successively larger cluster

- Divisive Algorithm
  - Begins with the whole set and proceed to divide it into successive smaller clusters


Agglomerative
p, q, r, s, t
p, q
r, s, t
s, t
p
q
r
s
t
Divisive

Agglomerative HC

STEP 1: Make each data point a single-point cluster ➡ That forms N clusters

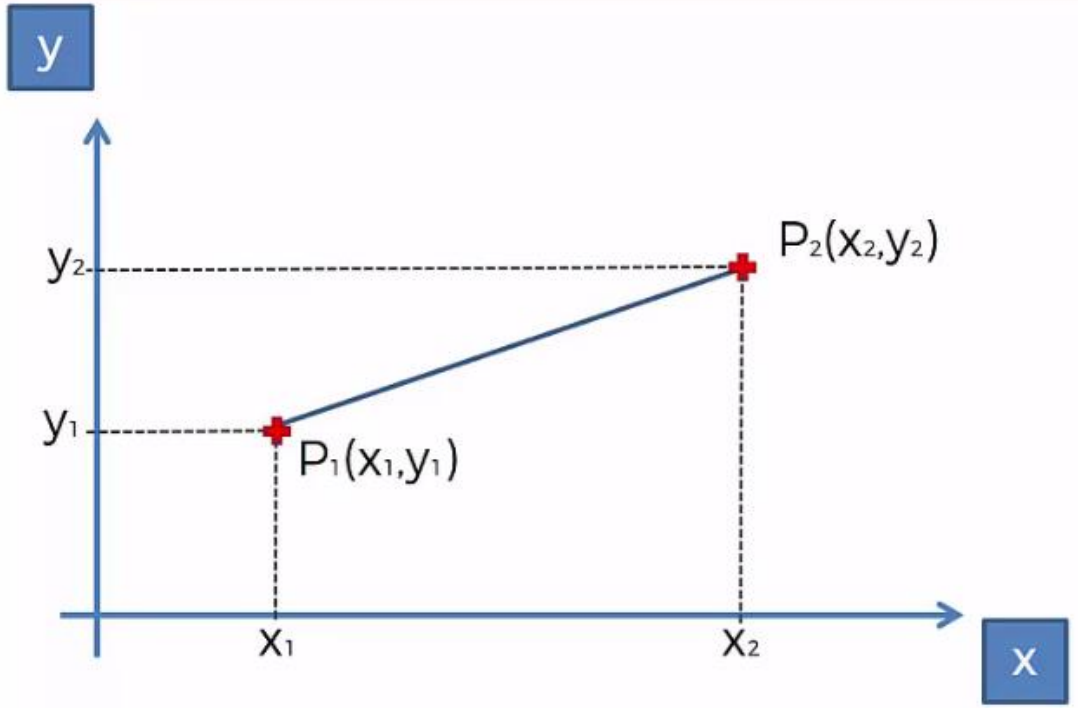
STEP 2: Take the two closest data points and make them one cluster ➡ That forms N-1 clusters

STEP 3: Take the two closest clusters and make them one cluster ➡ That forms N - 2 clusters

STEP 4: Repeat STEP 3 until there is only one cluster

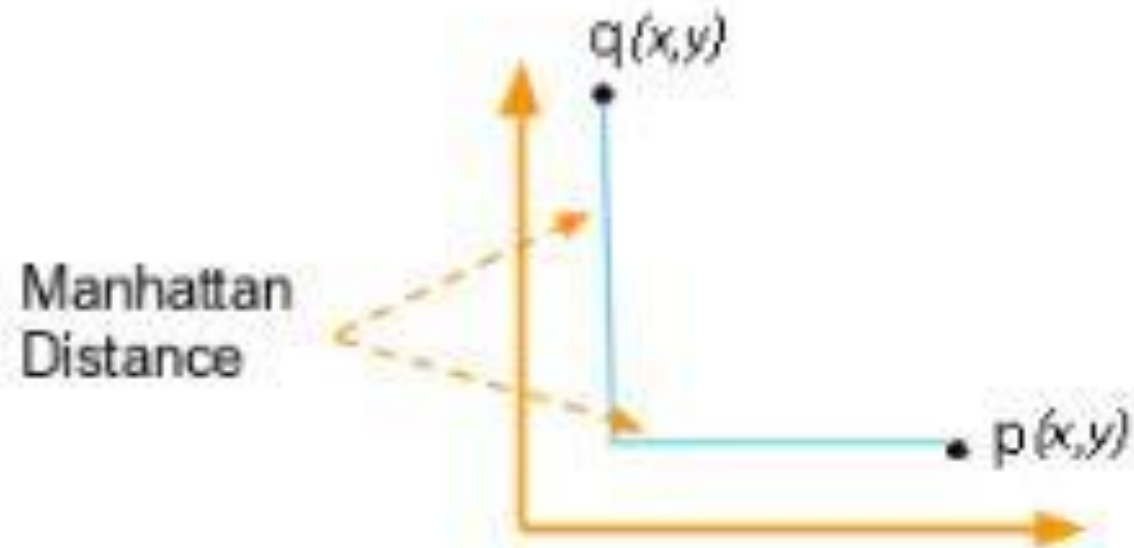
FIN

Euclidean Distance



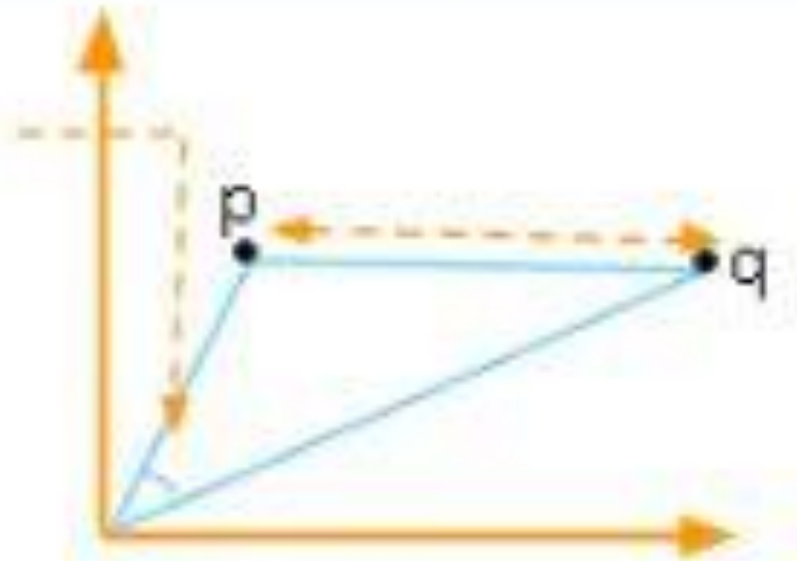
Euclidean Distance between P_1 and $P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

$$\mathbf{d} = \sum_{i=1}^{n} | q_x - p_x | + | q_y - p_y |$$

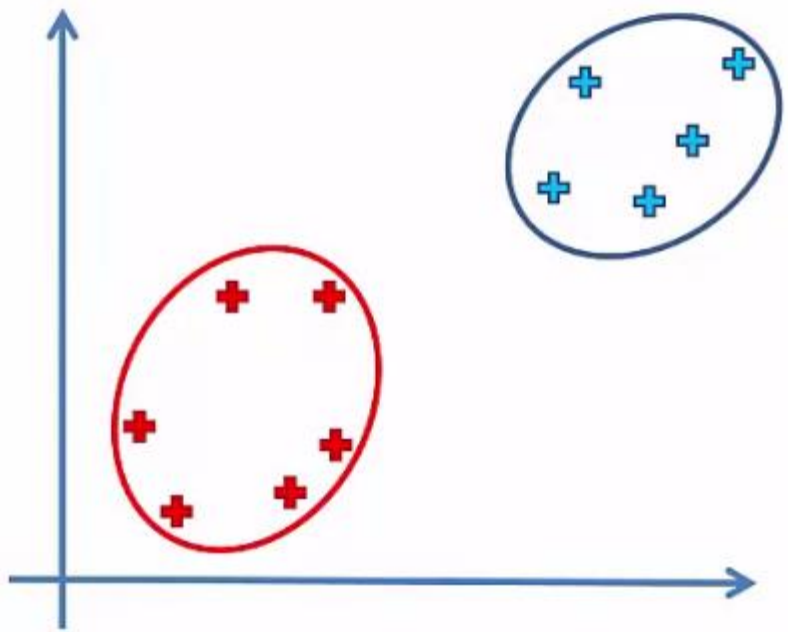


$$\mathbf{d} = \frac{\sum_{i=0}^{n-1} q_i - p_i}{\sum_{i=0}^{n-1} (q_i)^2 \times \sum_{i=0}^{n-1} (p_i)^2}$$

Cosine
Distance

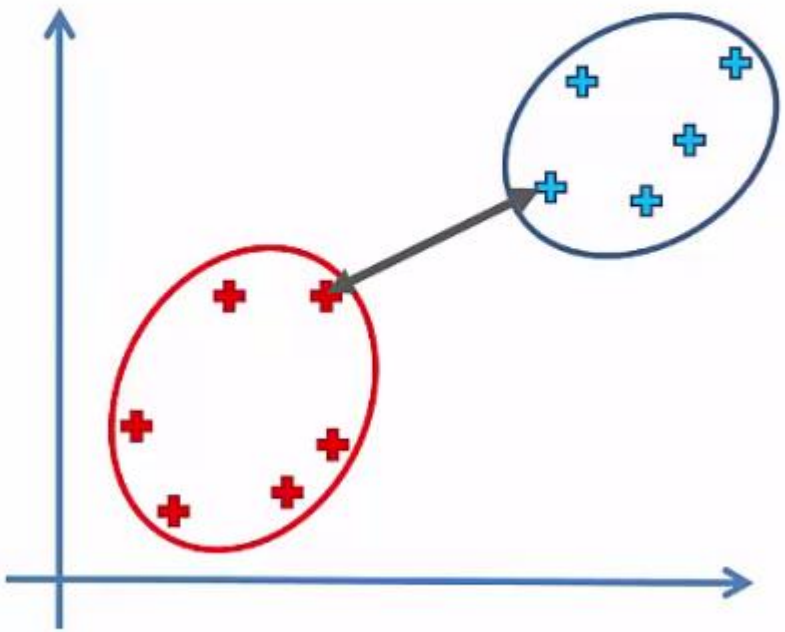


Distance Between Clusters



Distance Between Two Clusters:

Distance Between Clusters

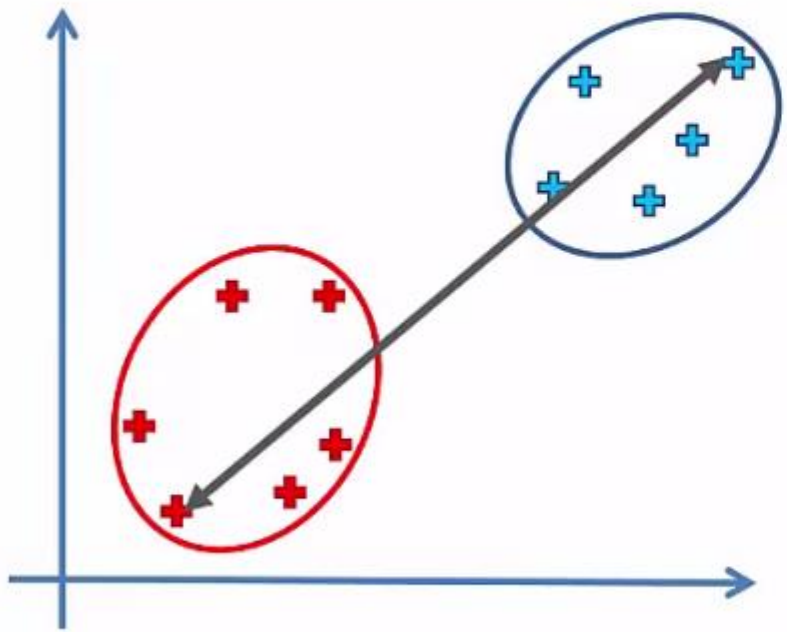


Distance Between Two Clusters:

- Option 1: Closest Points

Min (Single) Linkage

Distance Between Clusters

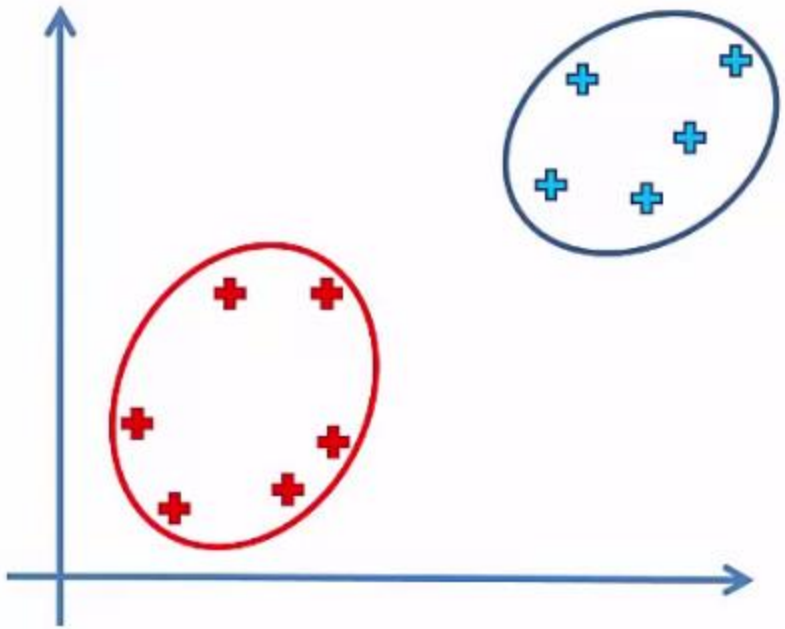


Distance Between Two Clusters:

- Option 1: Closest Points
- Option 2: Furthest Points

Max (Complete) Linkage

Distance Between Clusters

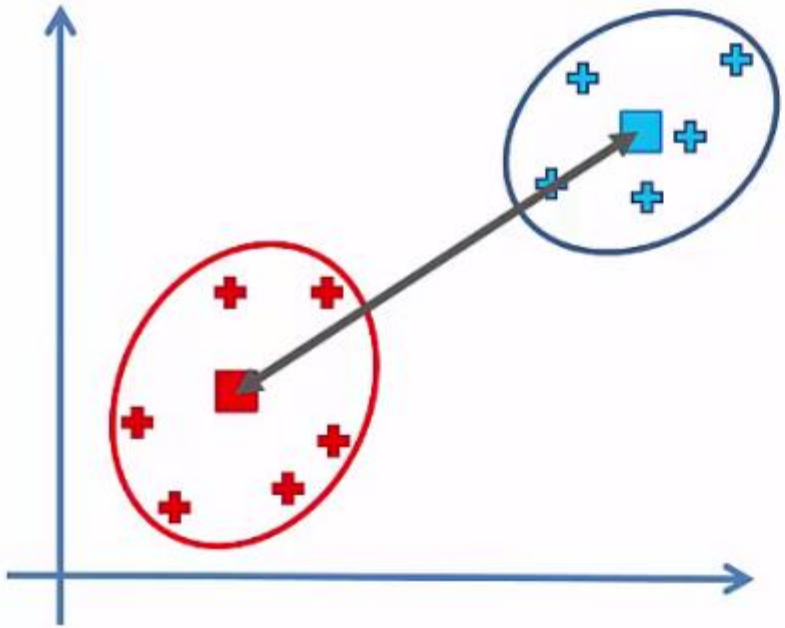


Distance Between Two Clusters:

- Option 1: Closest Points
- Option 2: Furthest Points
- Option 3: Average Distance

Average Linkage

Distance Between Clusters



Distance Between Two Clusters:

- Option 1: Closest Points
- Option 2: Furthest Points
- Option 3: Average Distance
- Option 4: Distance Between Centroids

Centroid Linkage

Within cluster variance: Ward Linkage

Agglomerative HC

Consider the following dataset of N = 6 data points



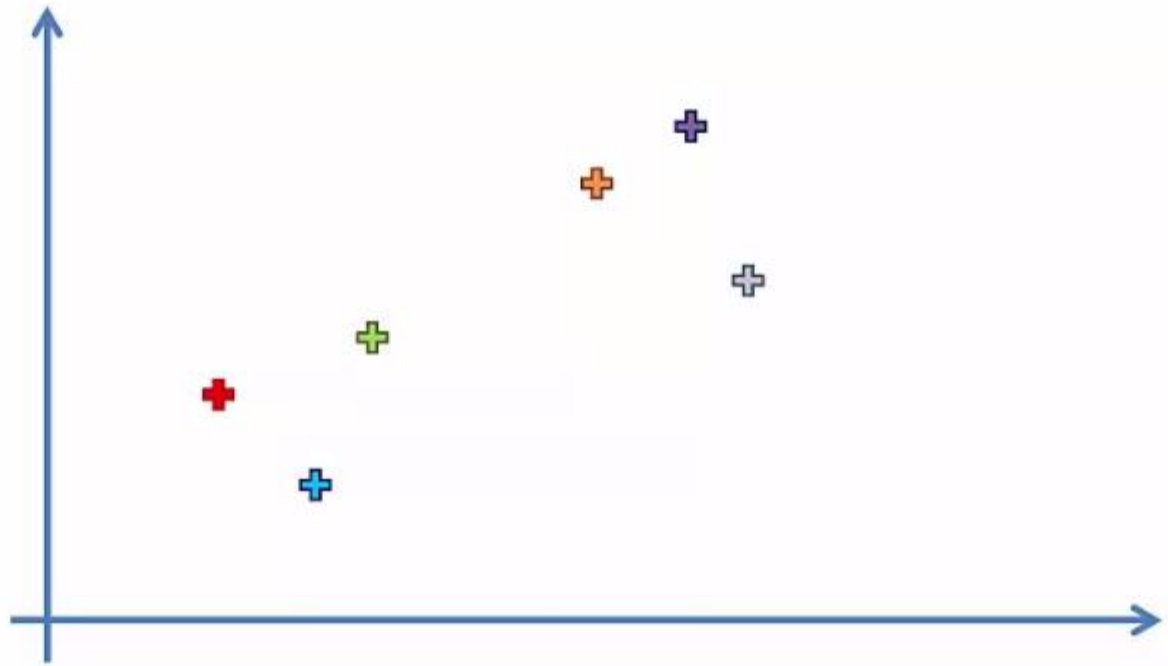
Agglomerative HC

STEP 1: Make each data point a single-point cluster ➡ That forms 6 clusters



Agglomerative HC

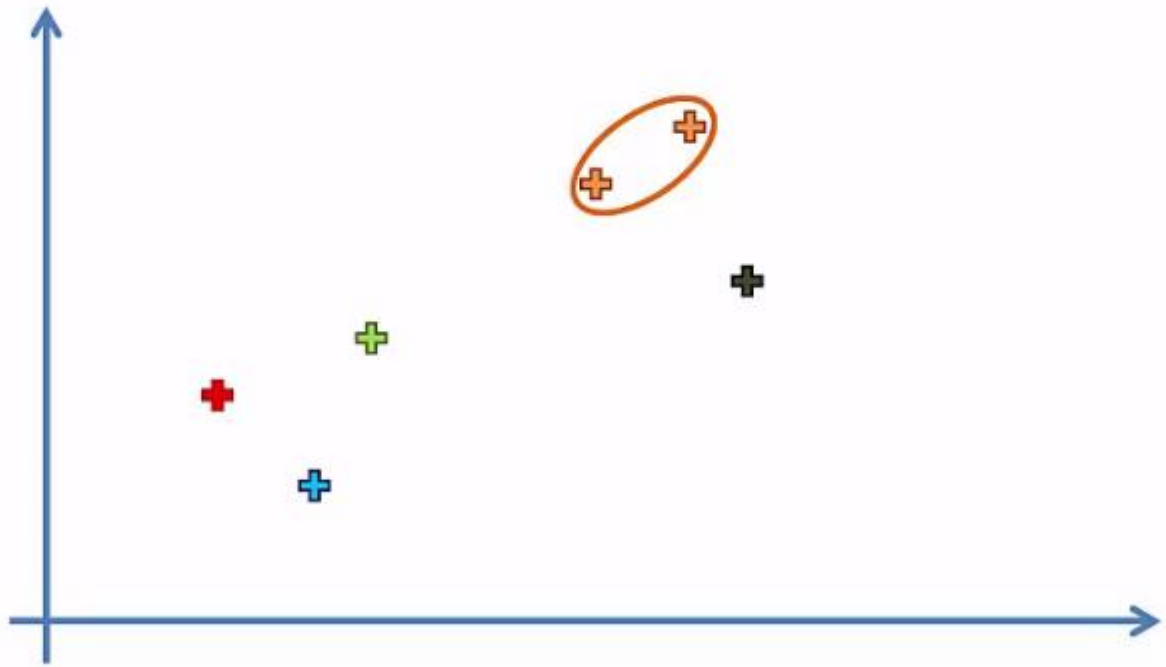
STEP 1: Make each data point a single-point cluster ➡ That forms 6 clusters



Agglomerative HC

STEP 2: Take the two closest data points and make them one cluster

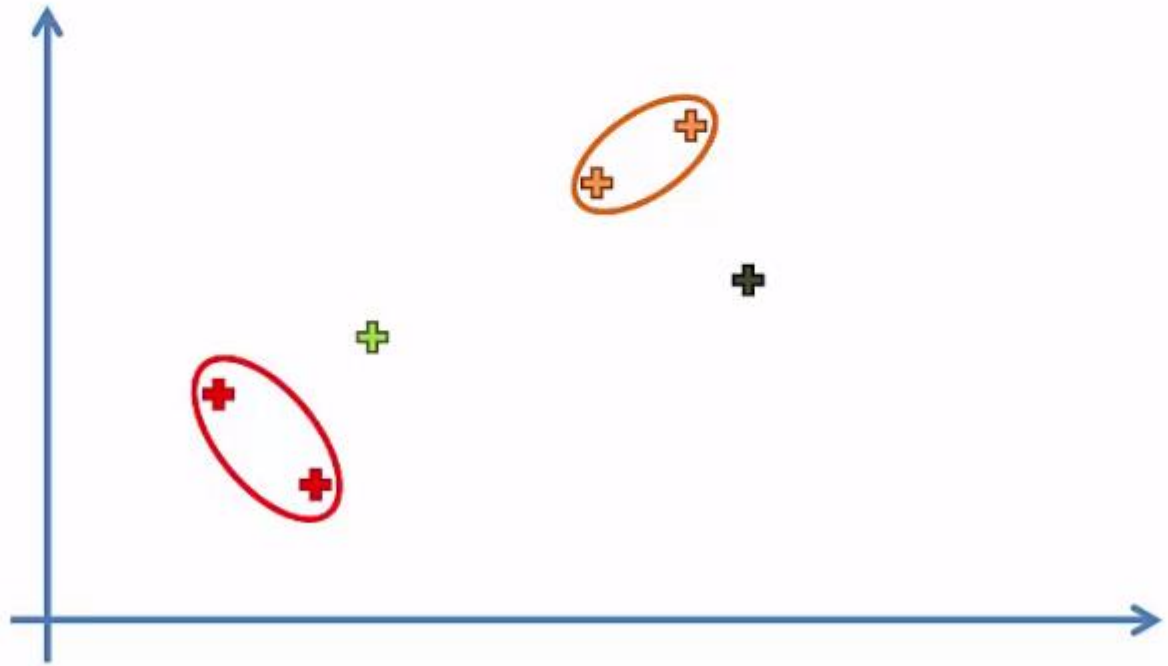
➡ That forms 5 clusters



Agglomerative HC

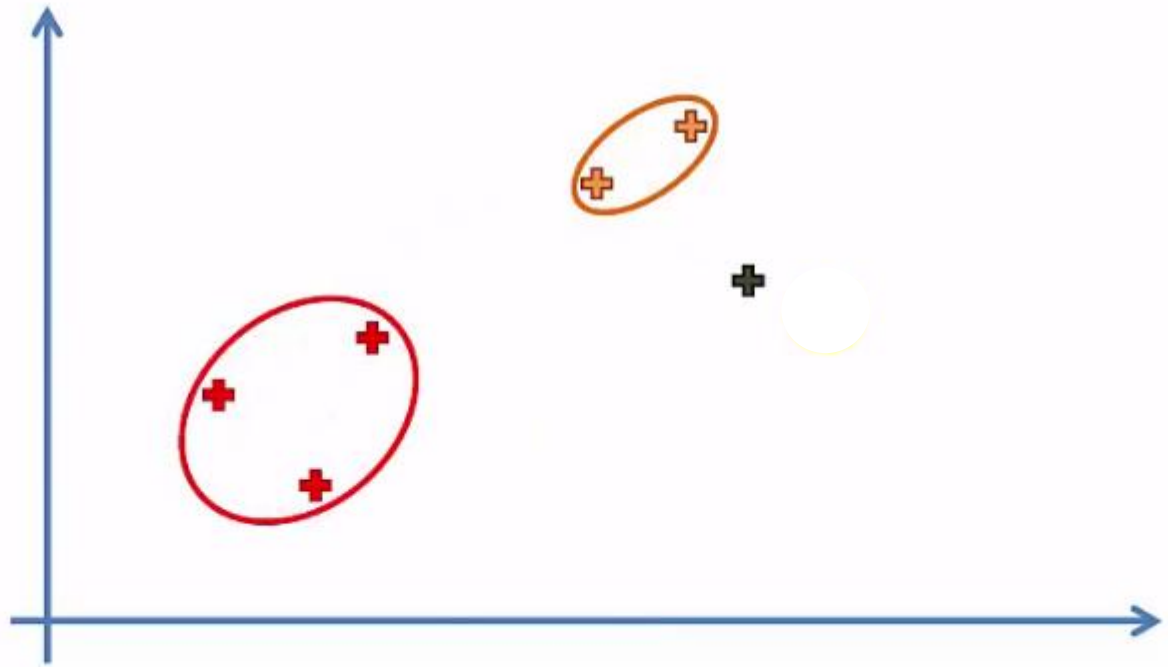
STEP 3: Take the two closest clusters and make them one cluster

➡ That forms 4 clusters



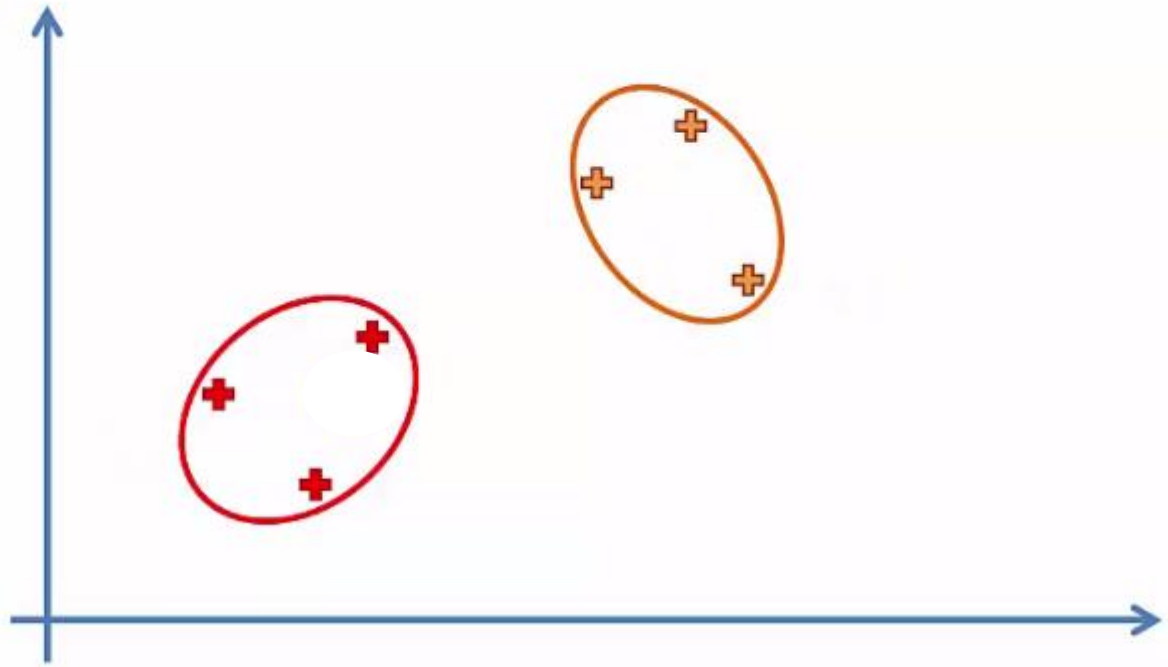
Agglomerative HC

STEP 4: Repeat STEP 3 until there is only one cluster



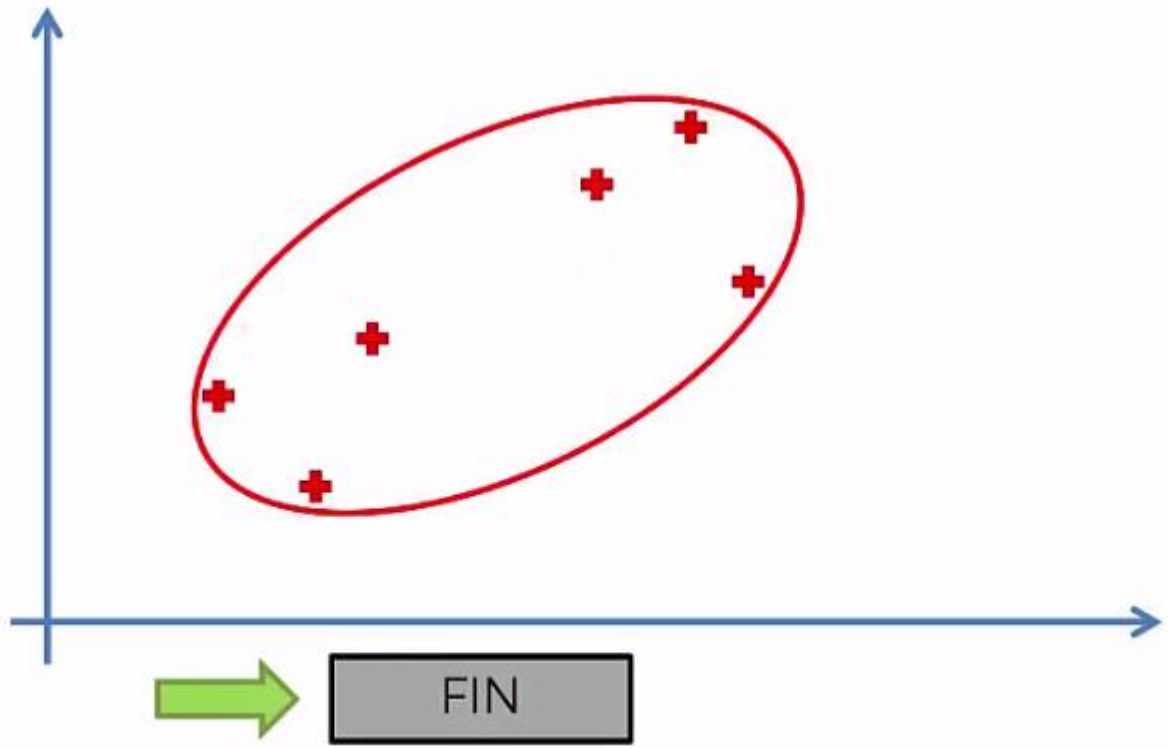
Agglomerative HC

STEP 4: Repeat STEP 3 until there is only one cluster



Agglomerative HC

STEP 4: Repeat STEP 3 until there is only one cluster

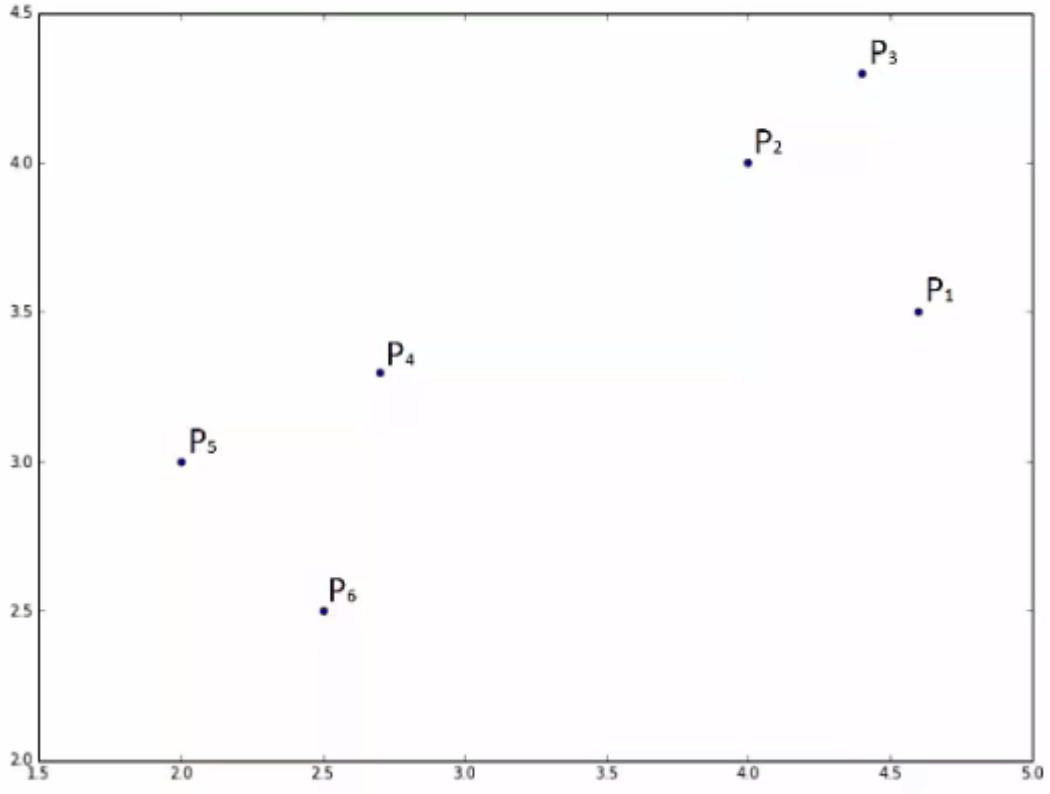


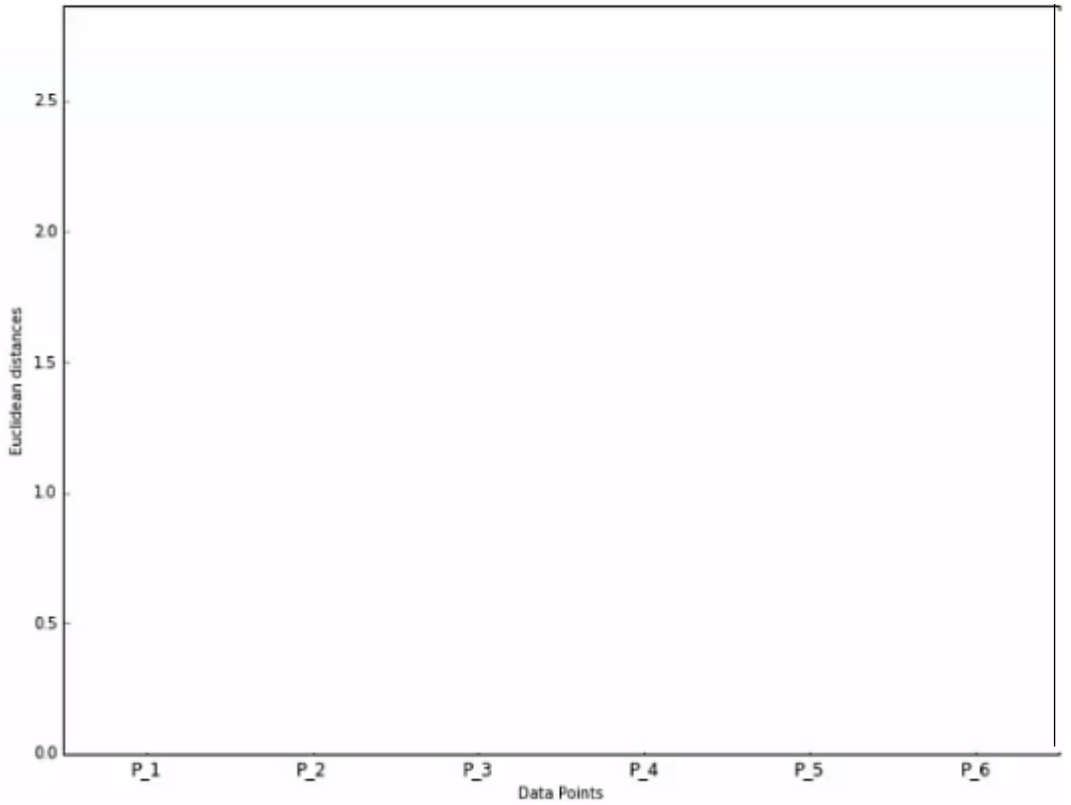
Hierarchical Clustering

Dendrograms

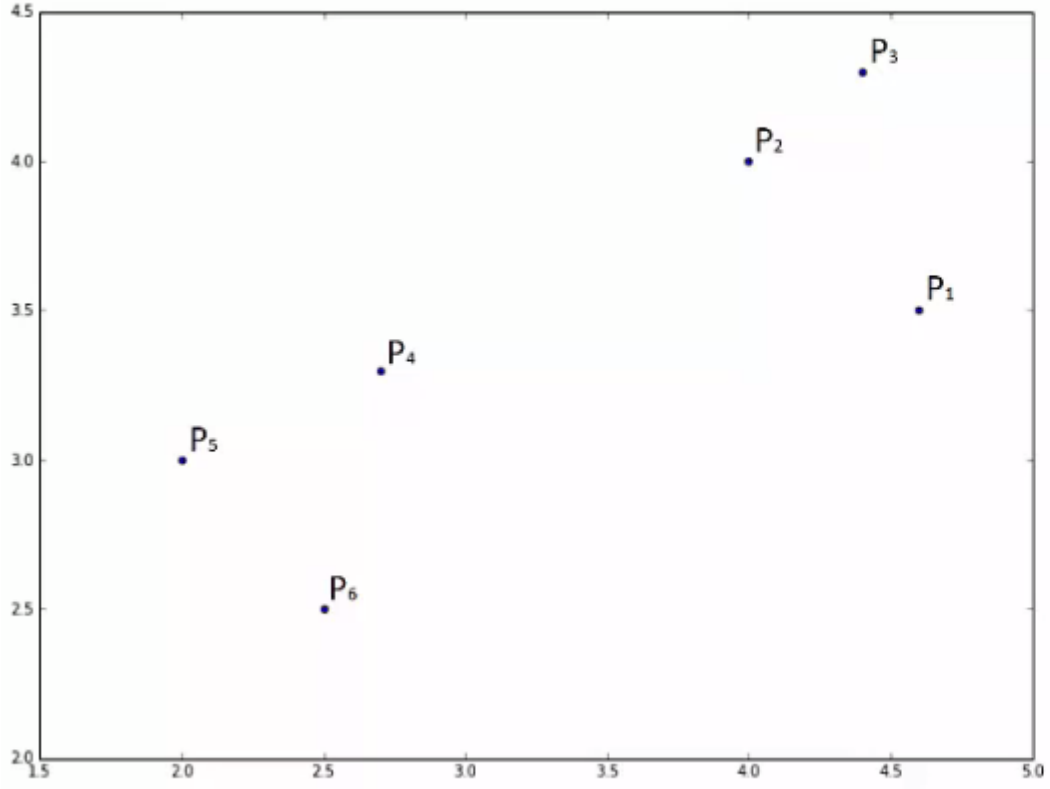
How do Dendrograms Works?

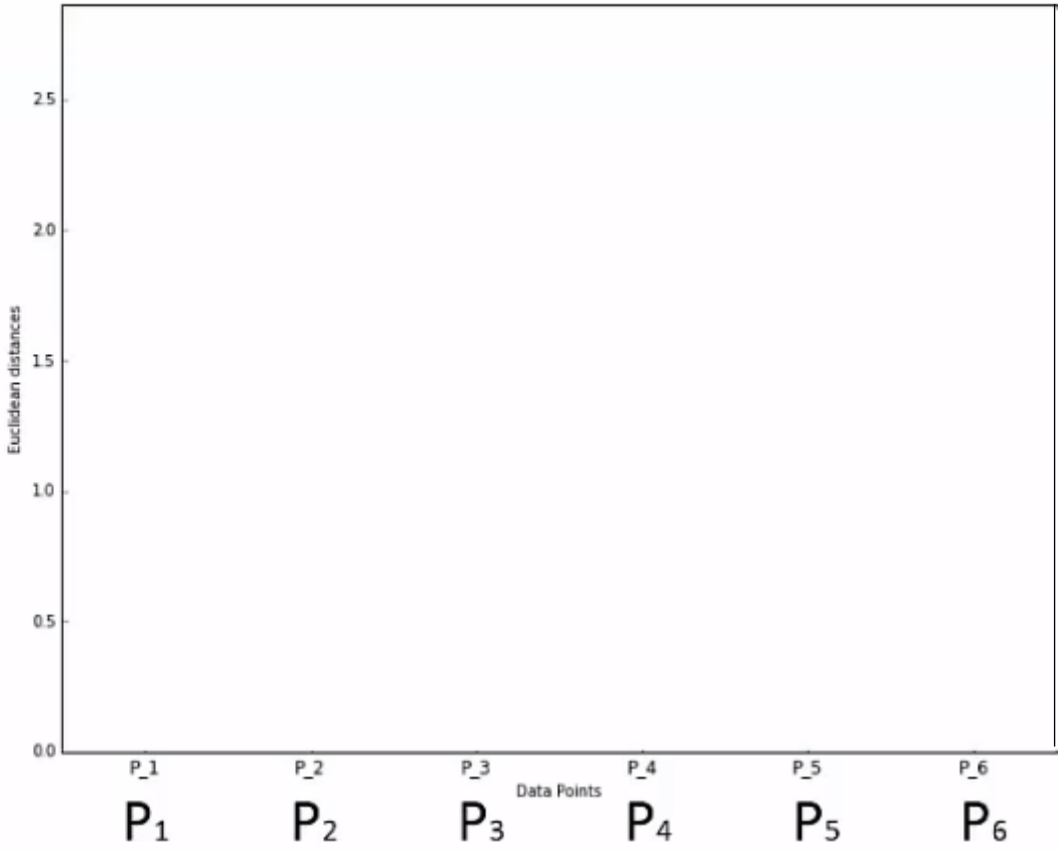
How do Dendrograms Works



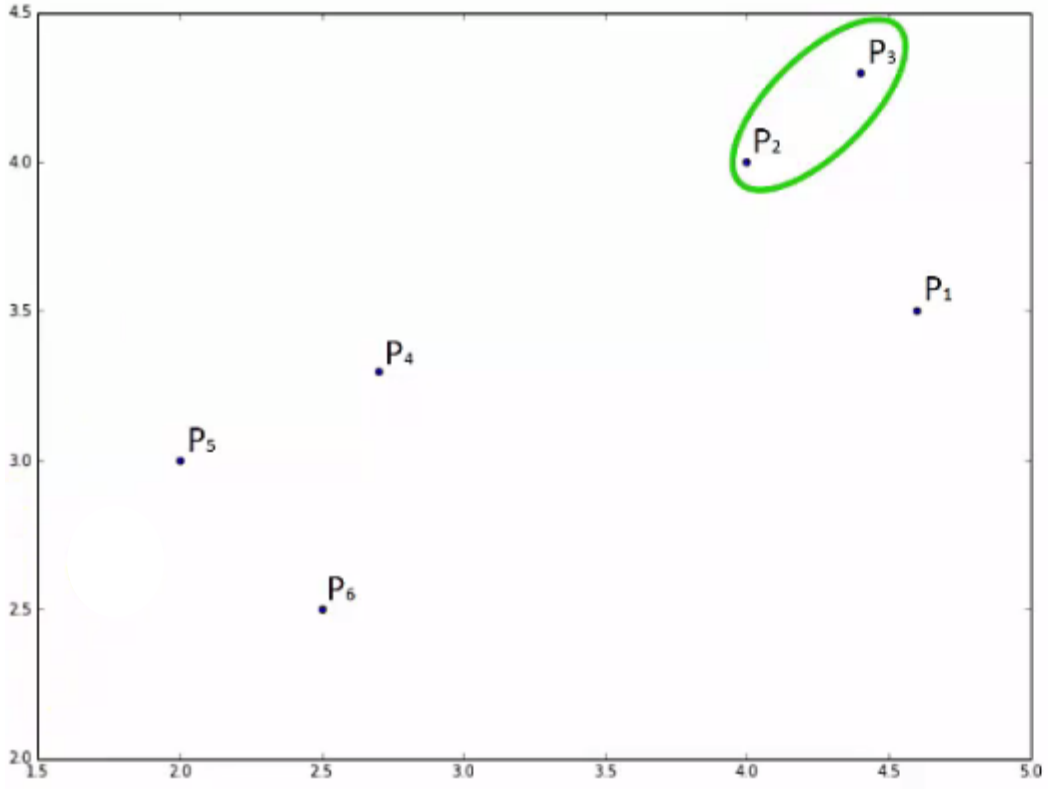


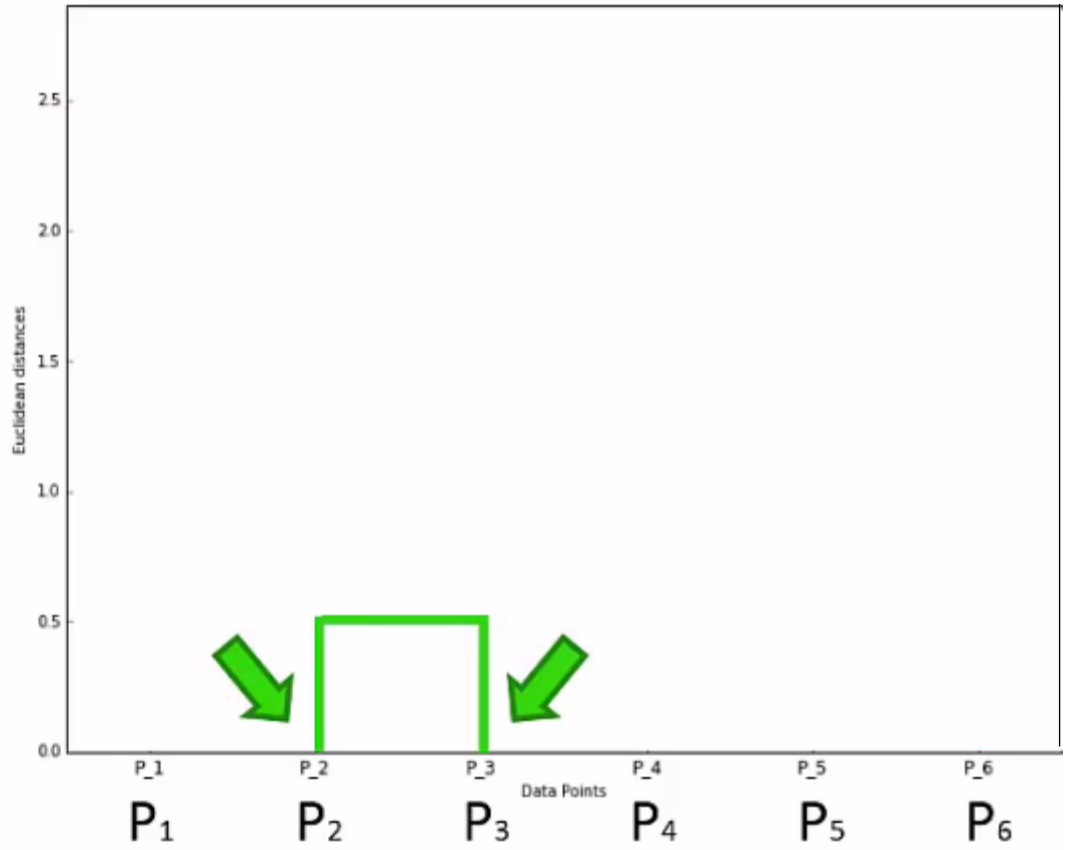
How do Dendrograms Works



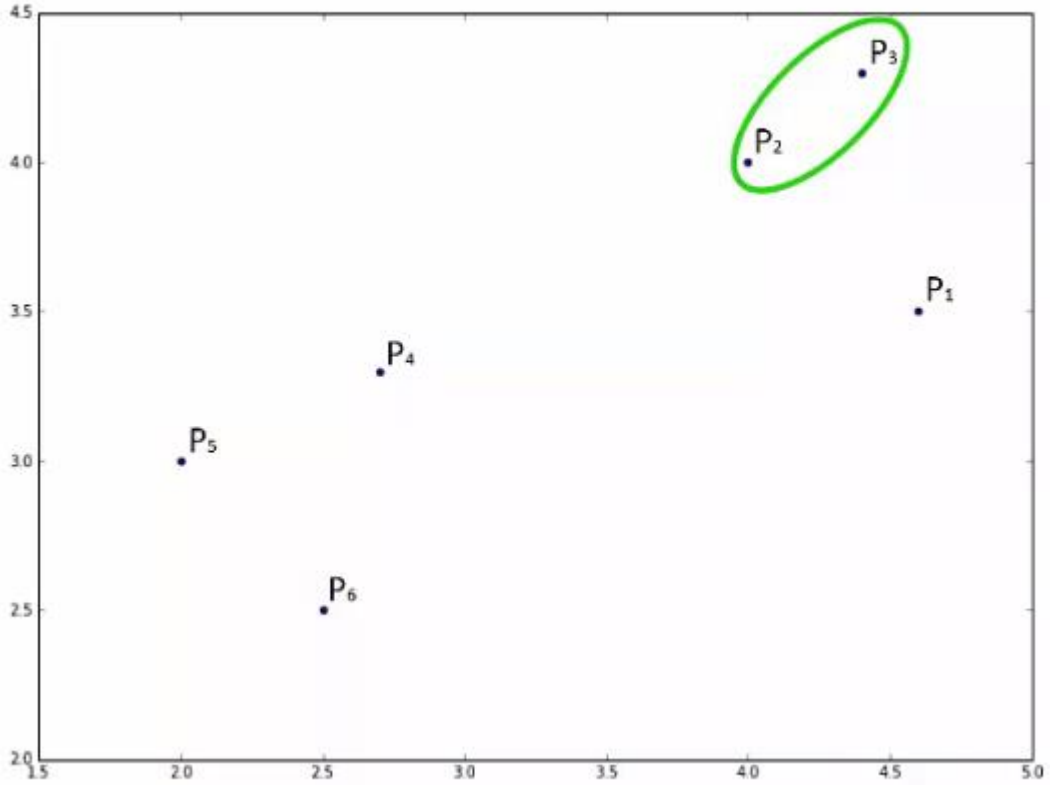


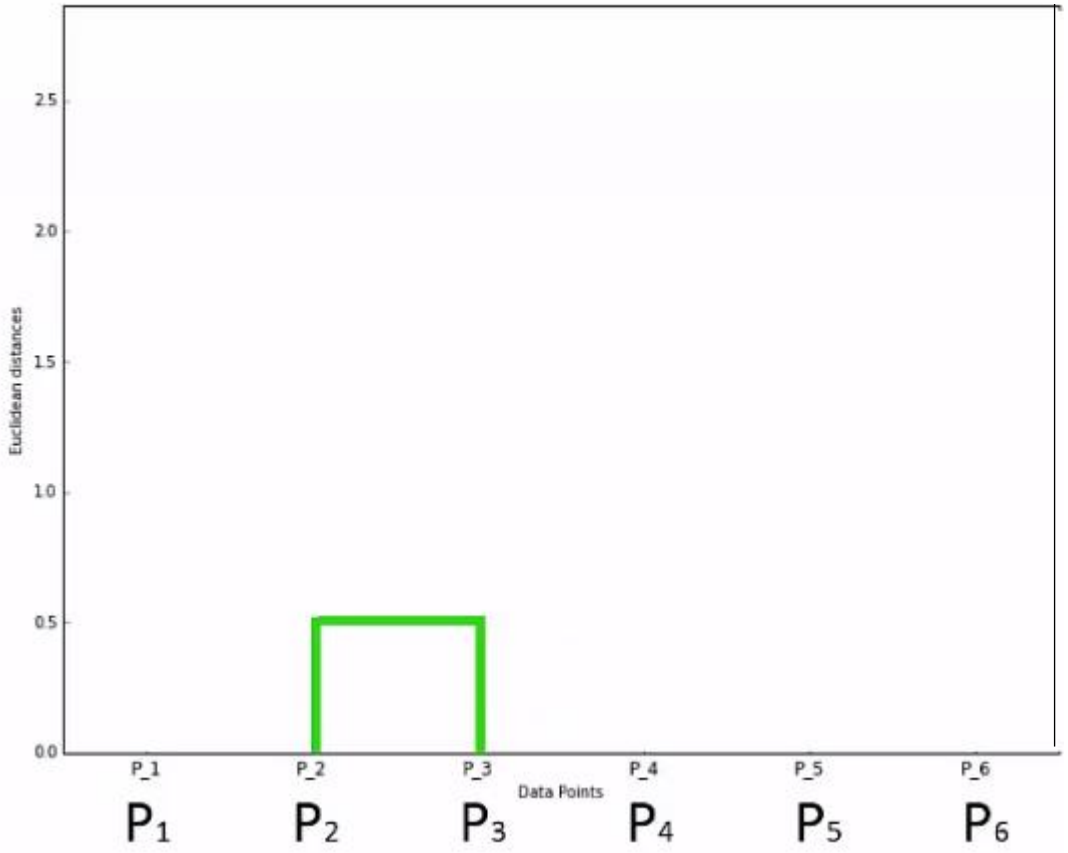
How do Dendrograms Works



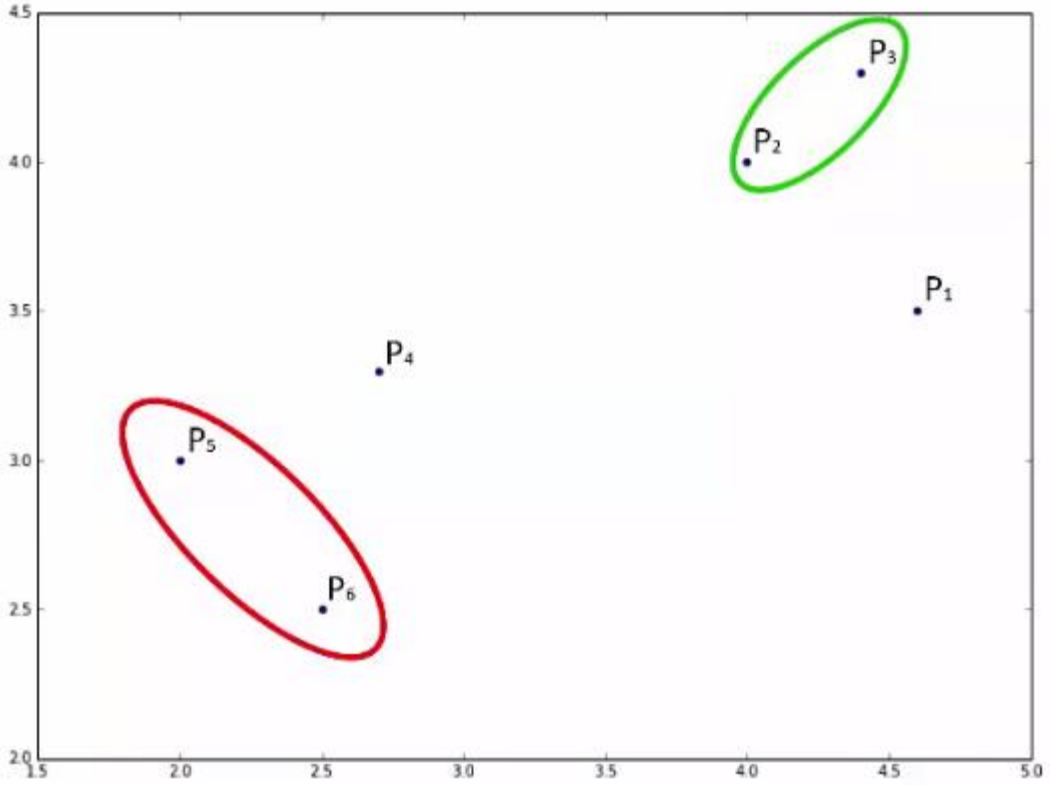


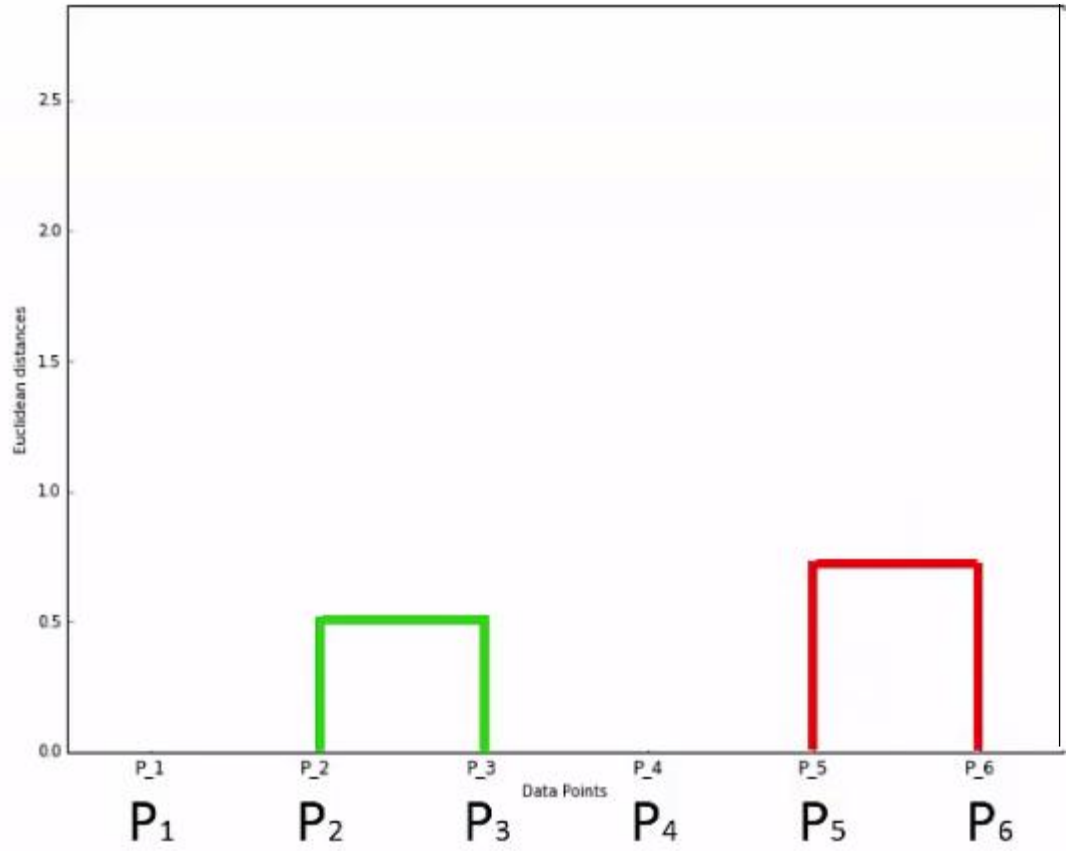
How do Dendrograms Works



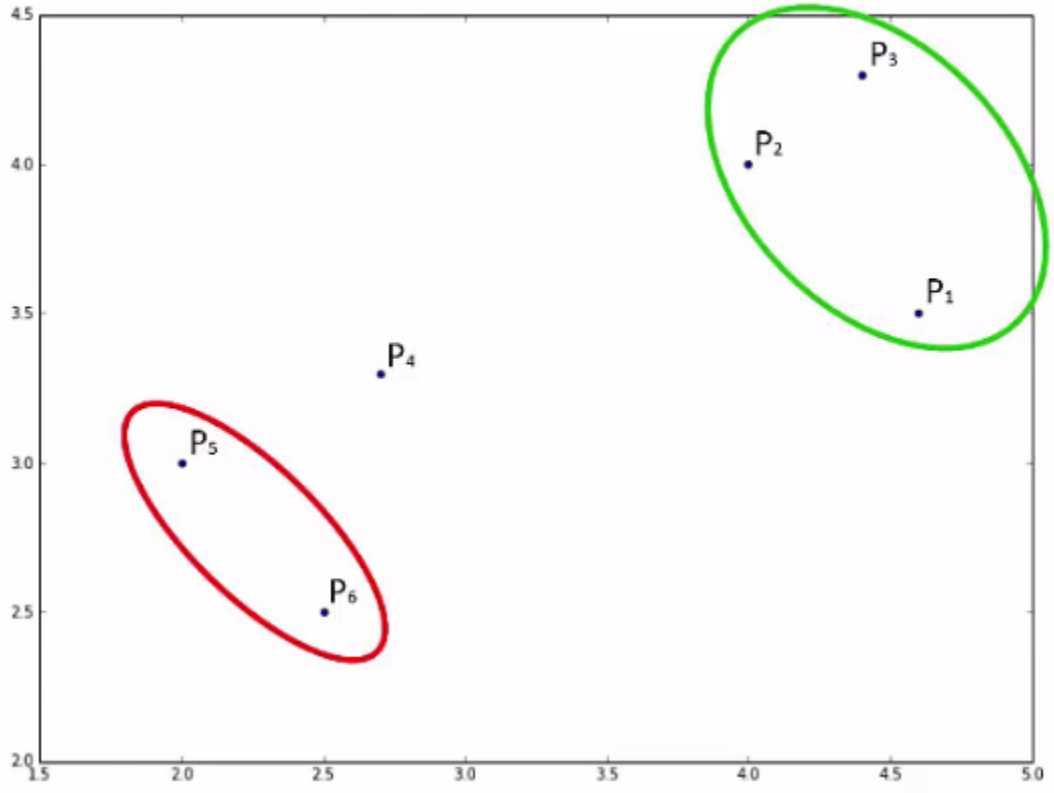


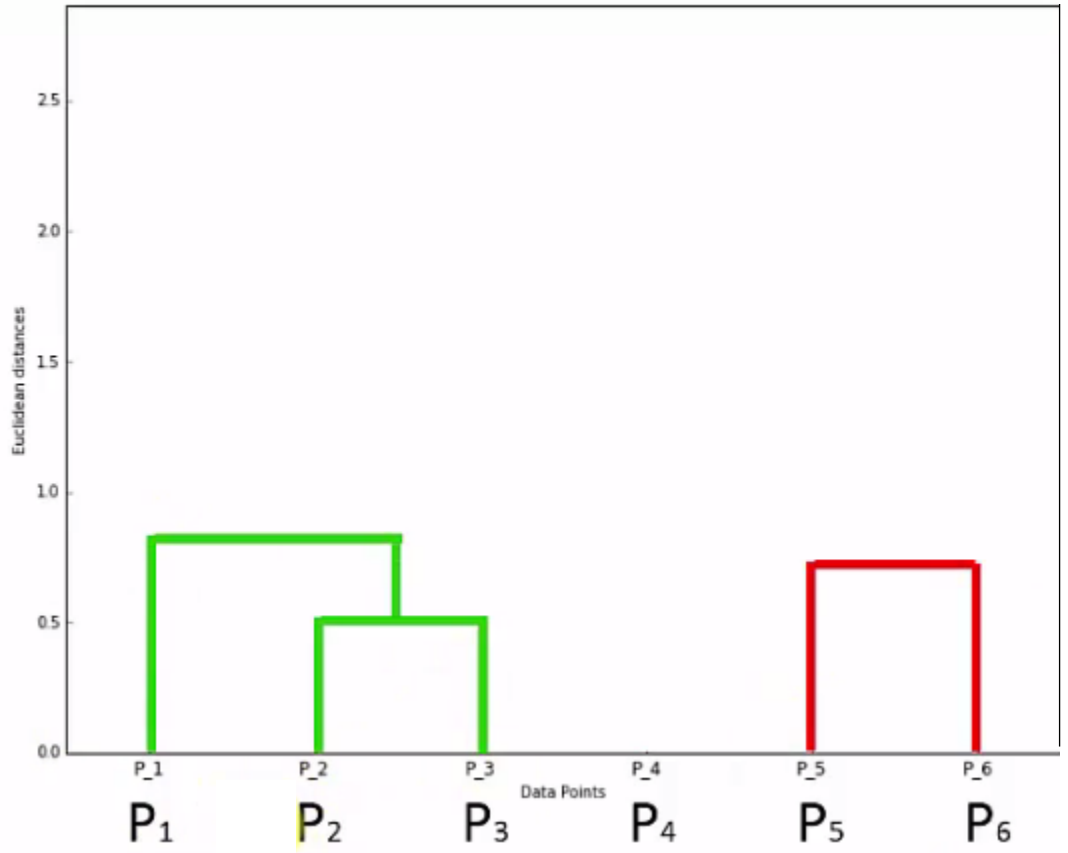
How do Dendrograms Works



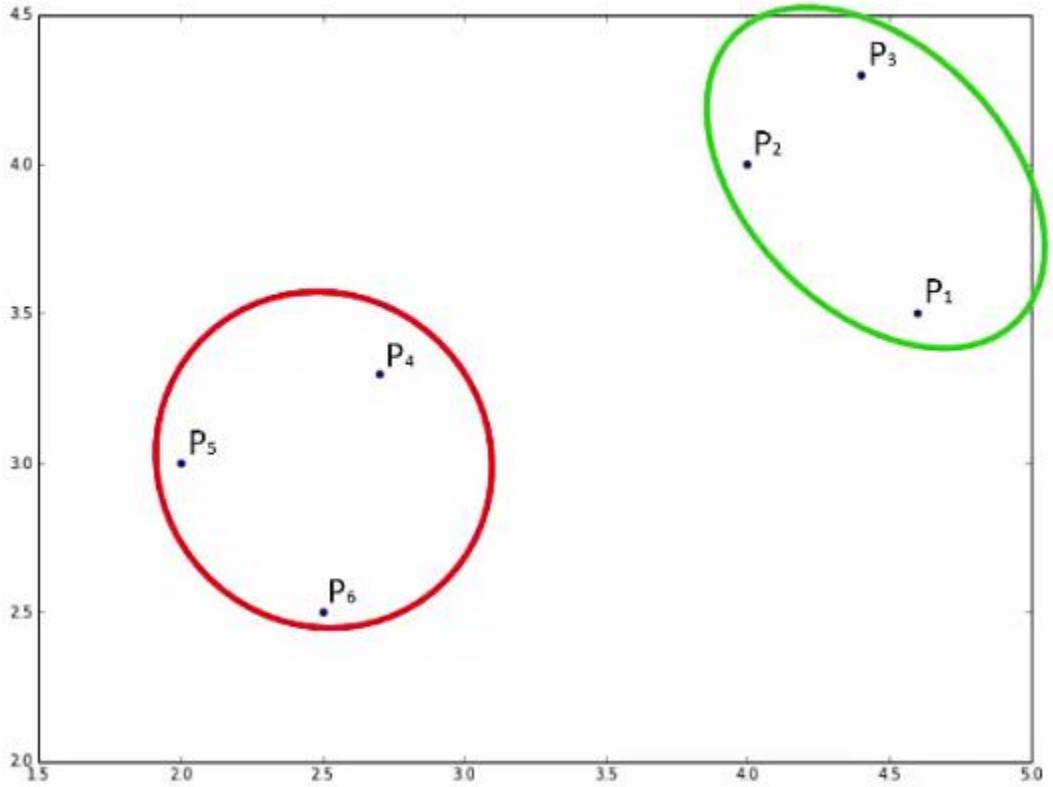


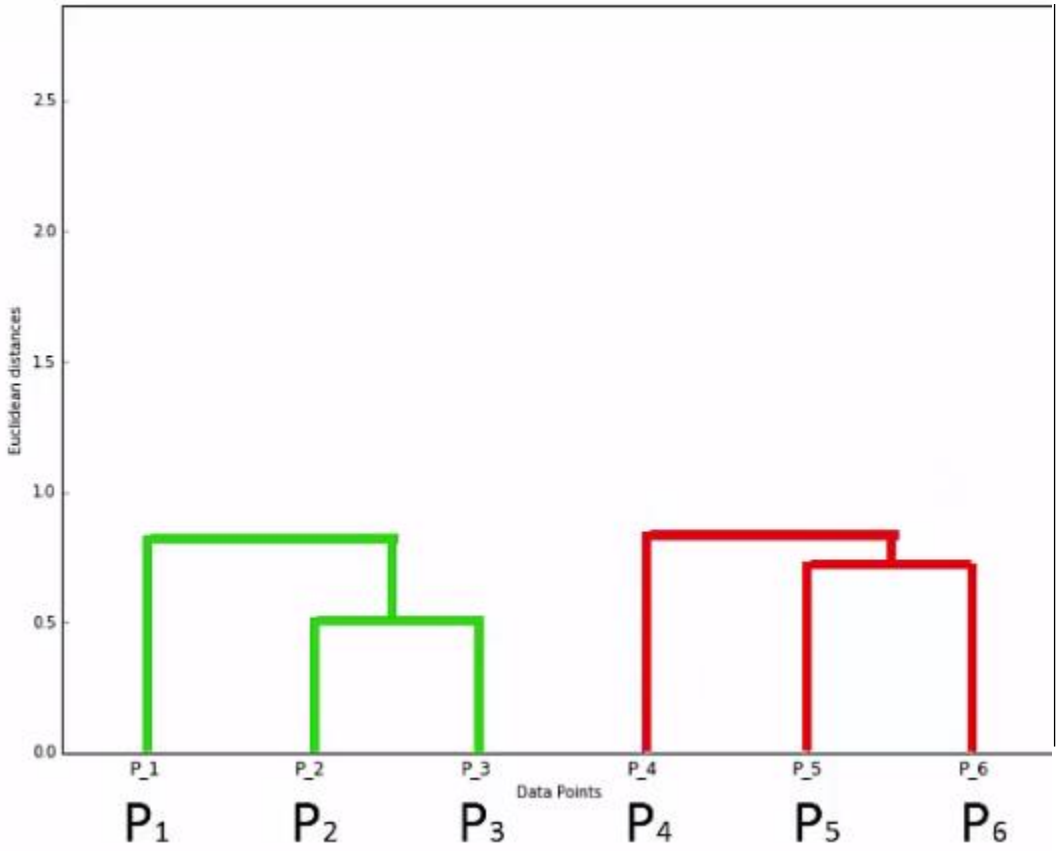
How do Dendrograms Works



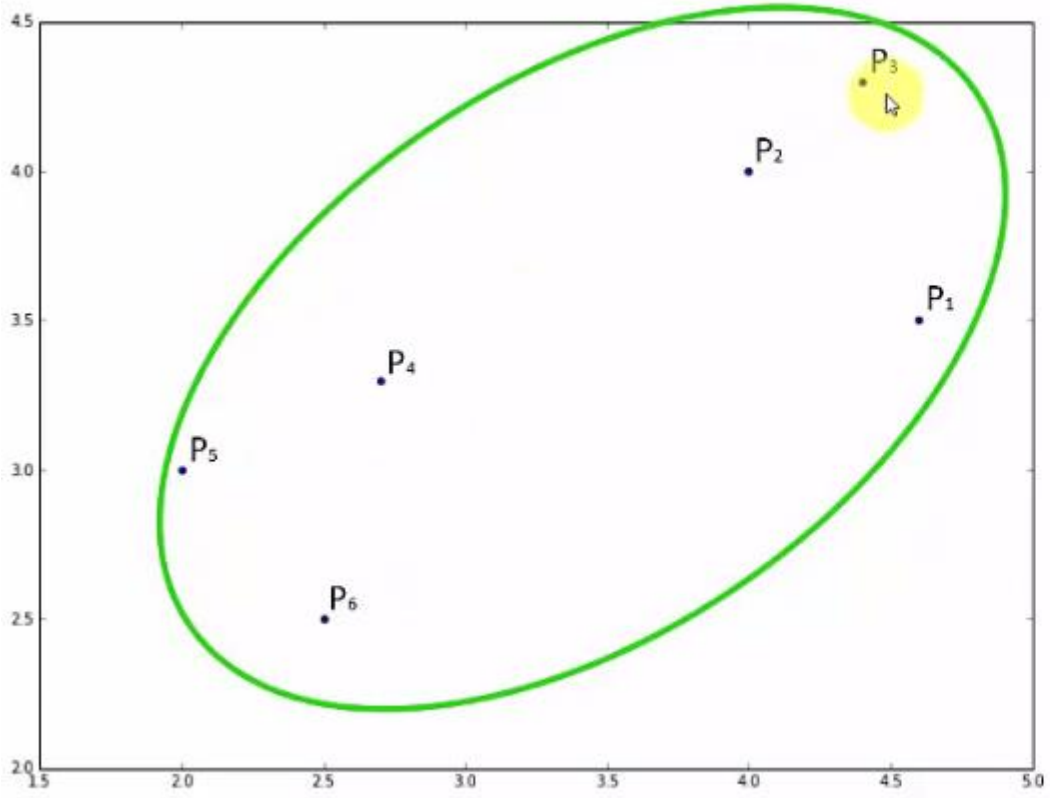


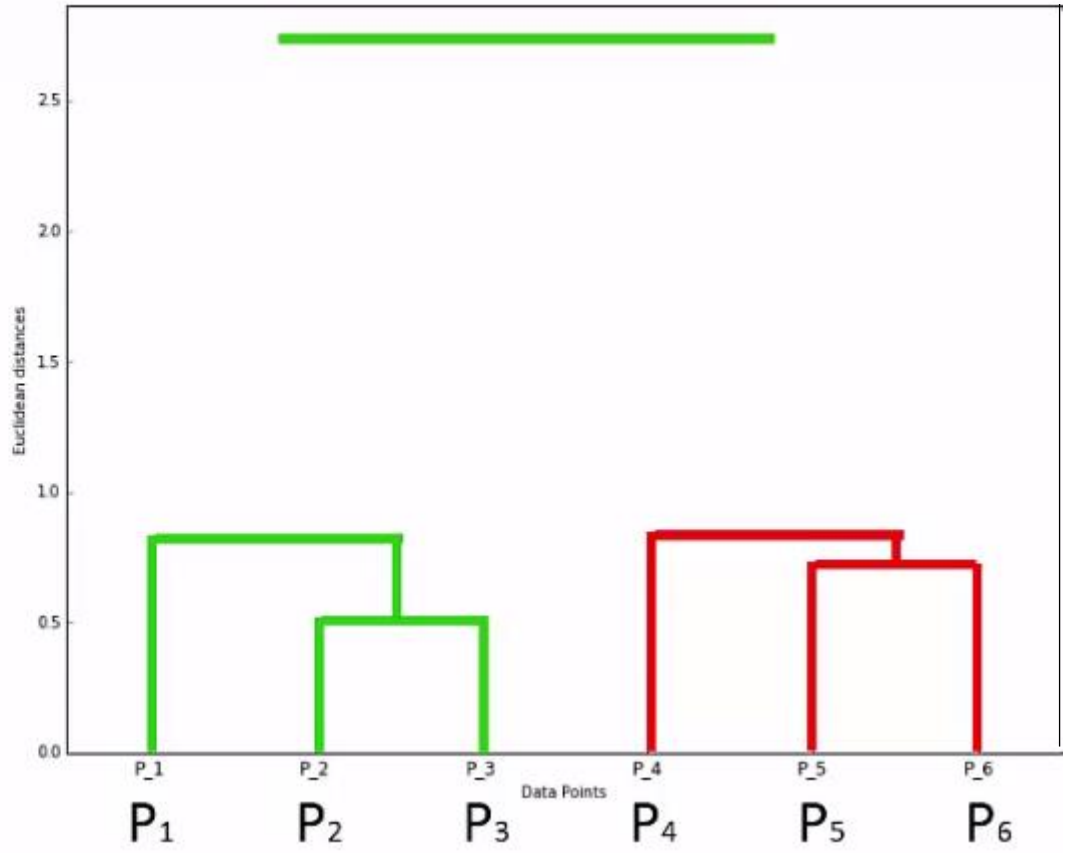
How do Dendrograms Works



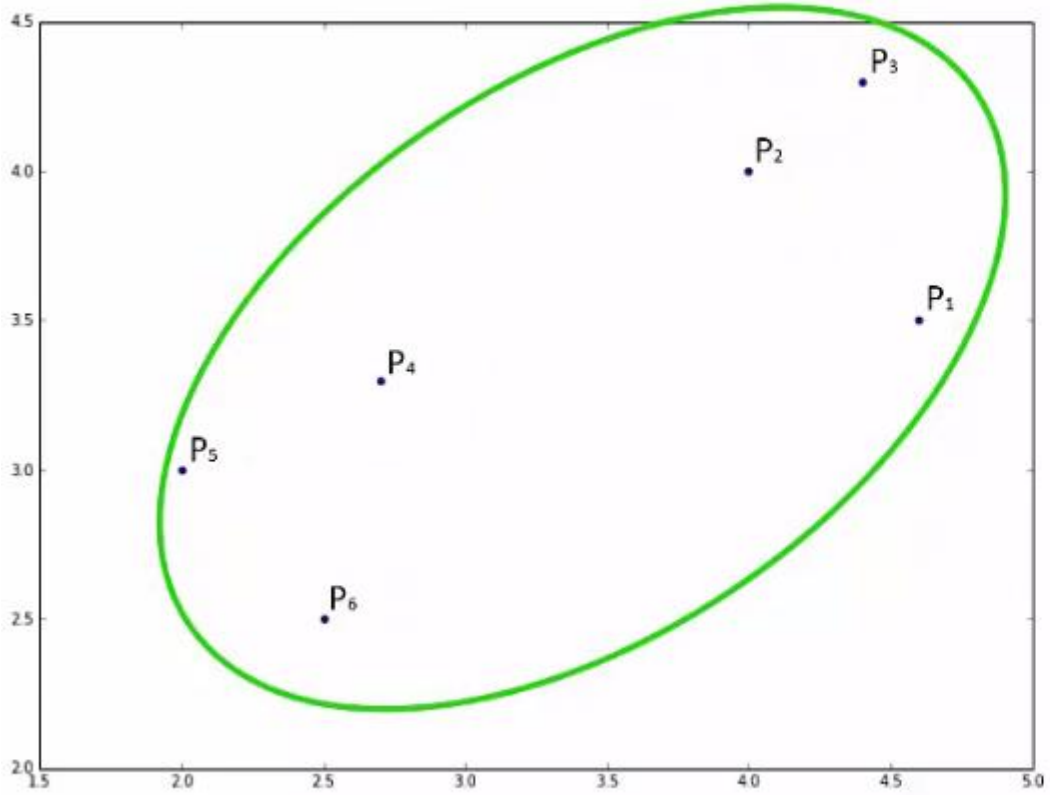


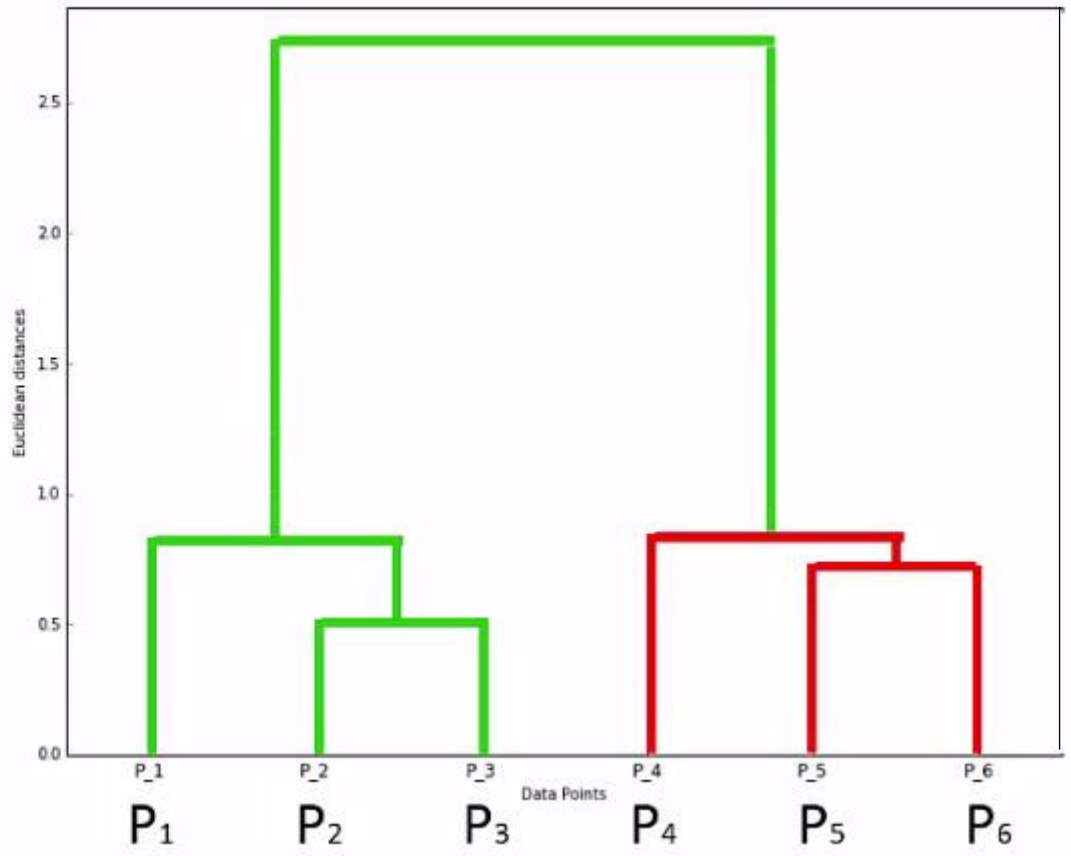
How do Dendrograms Works



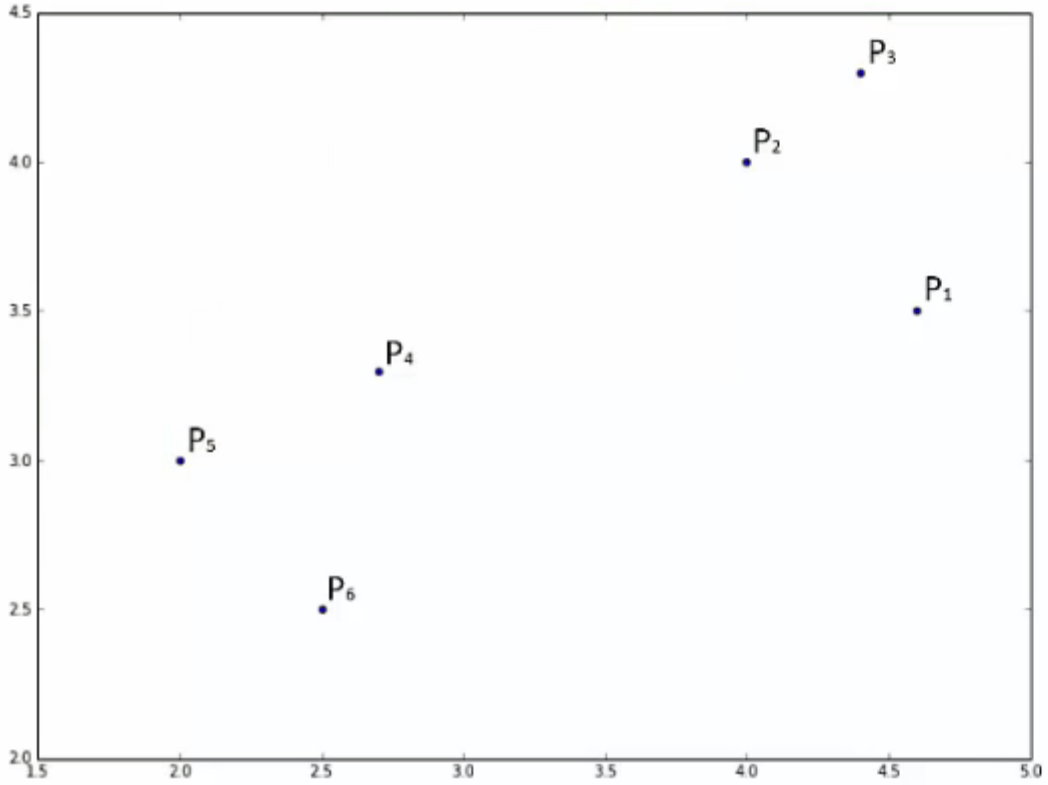


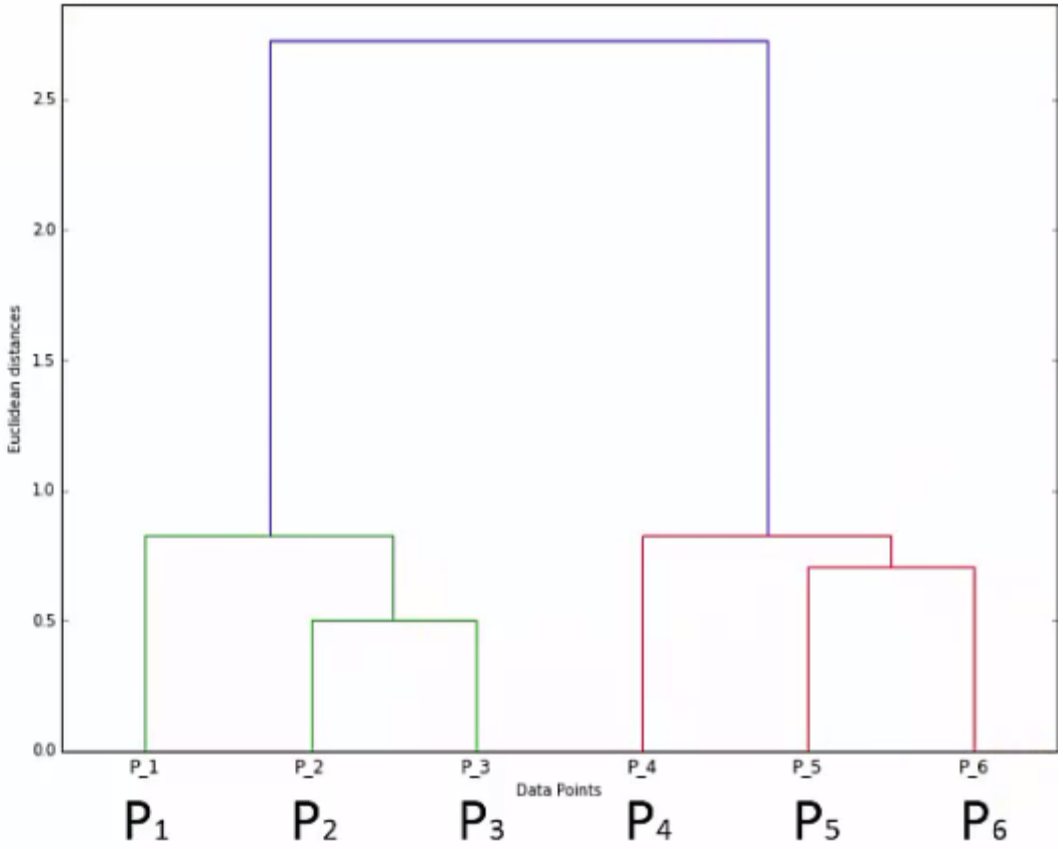
How do Dendrograms Works





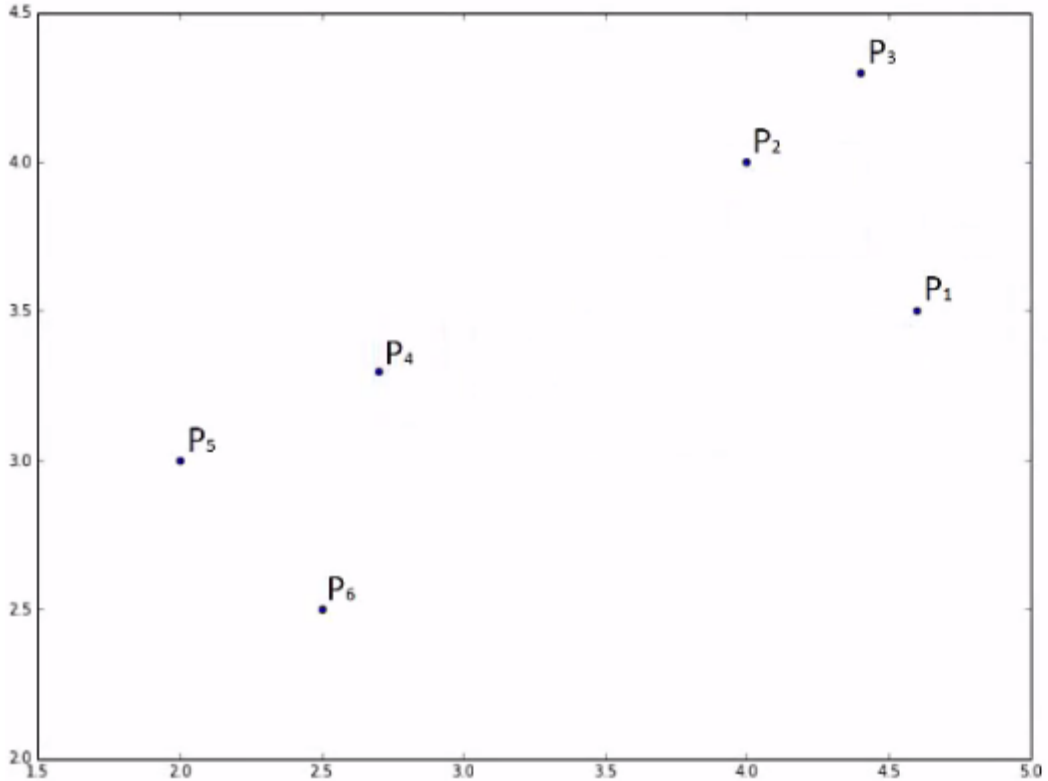
How do Dendrograms Works

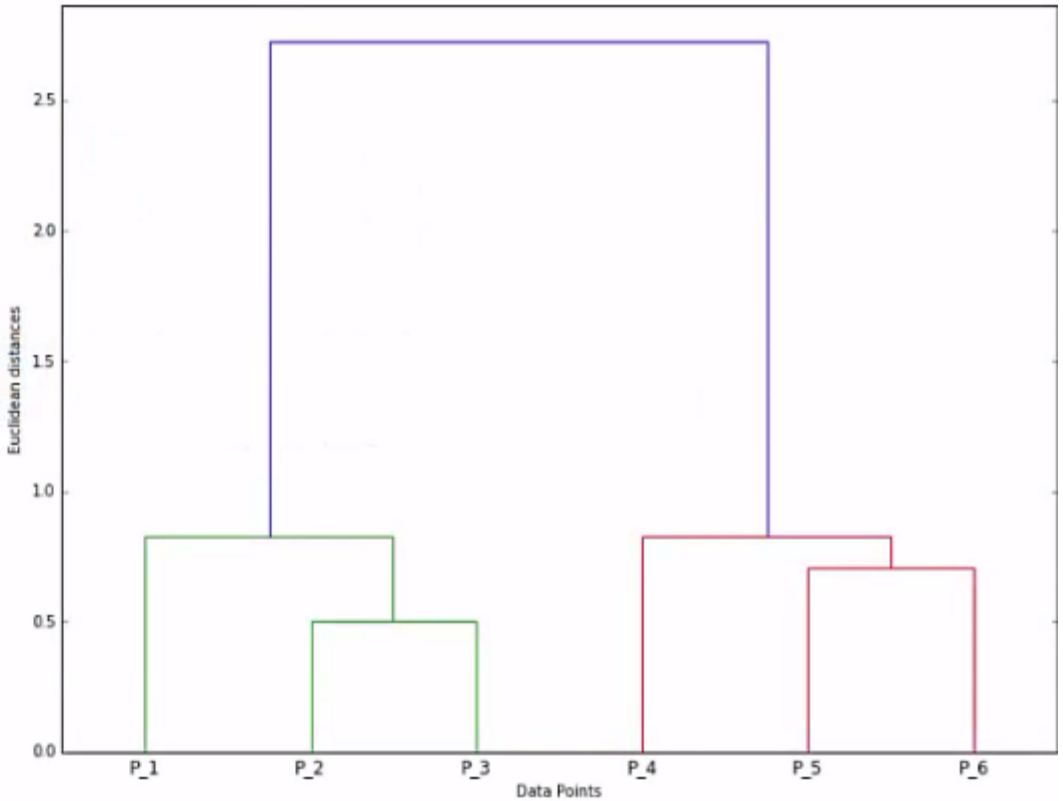




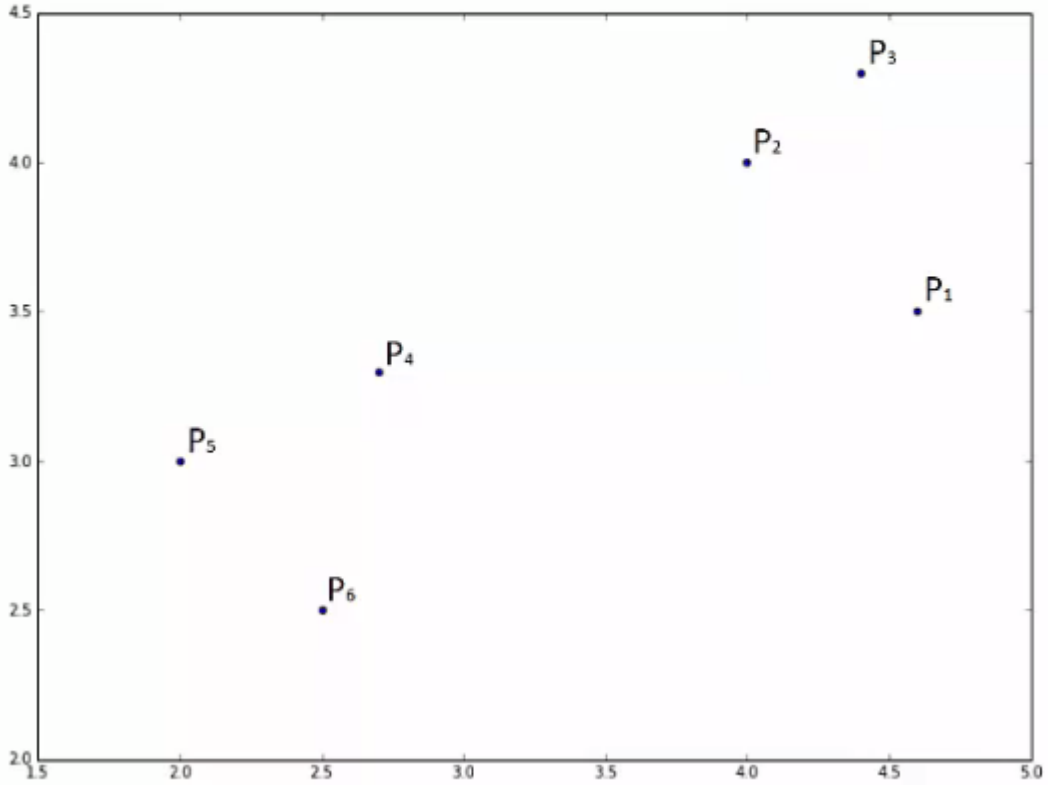
Using Dendrograms

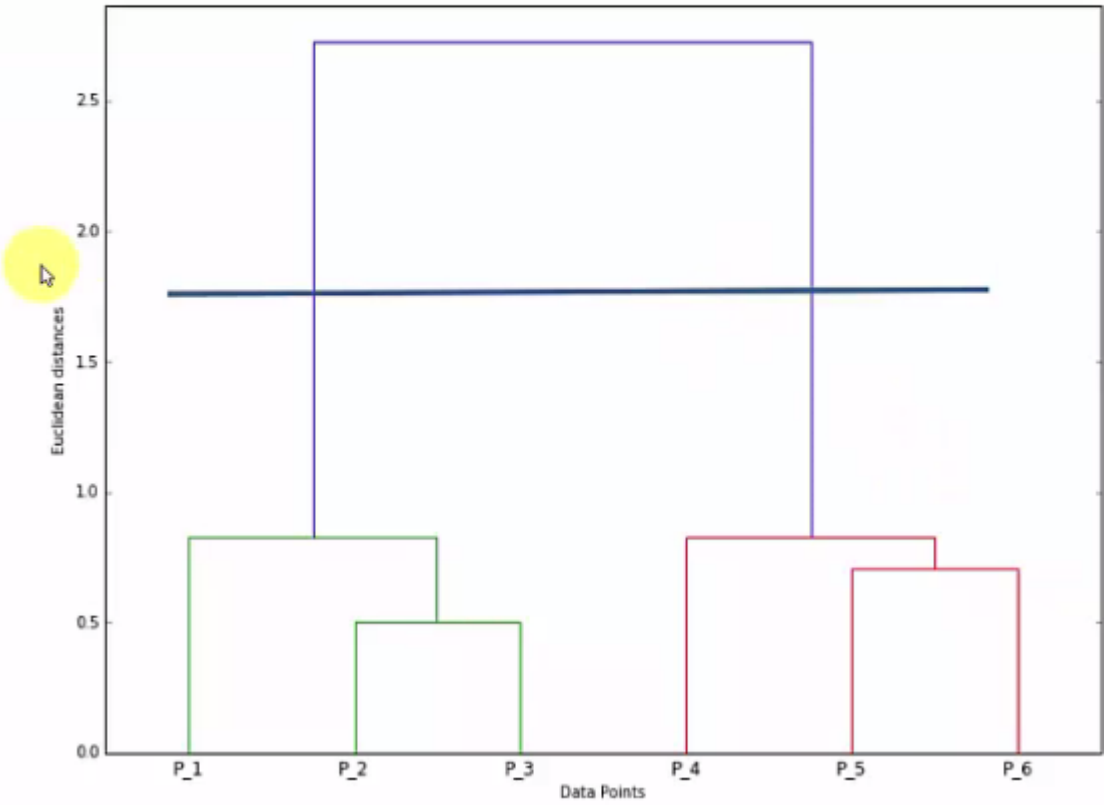
Dendrograms – Two Clusters



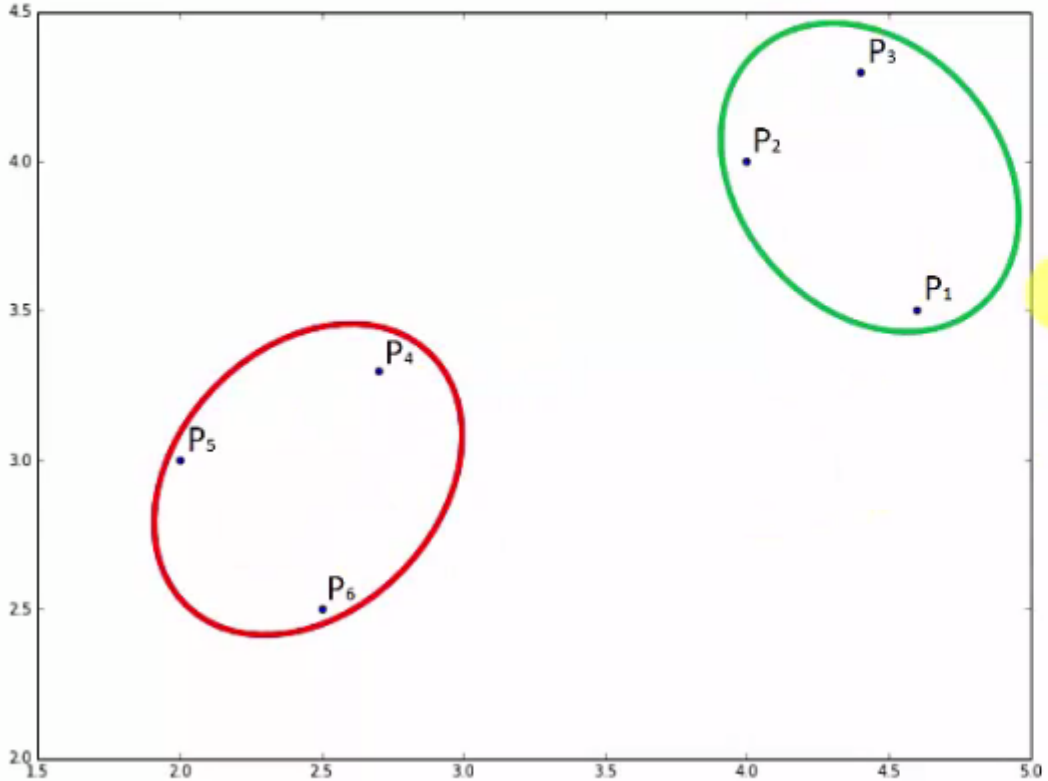


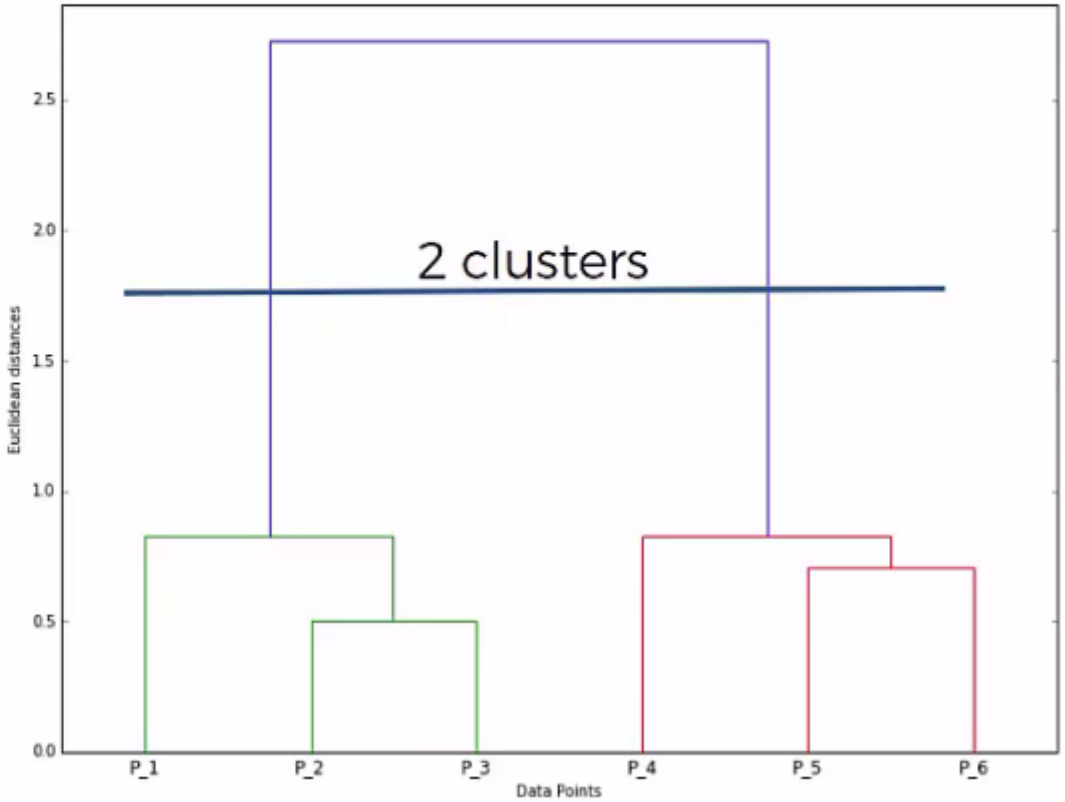
Dendrograms – Two Clusters



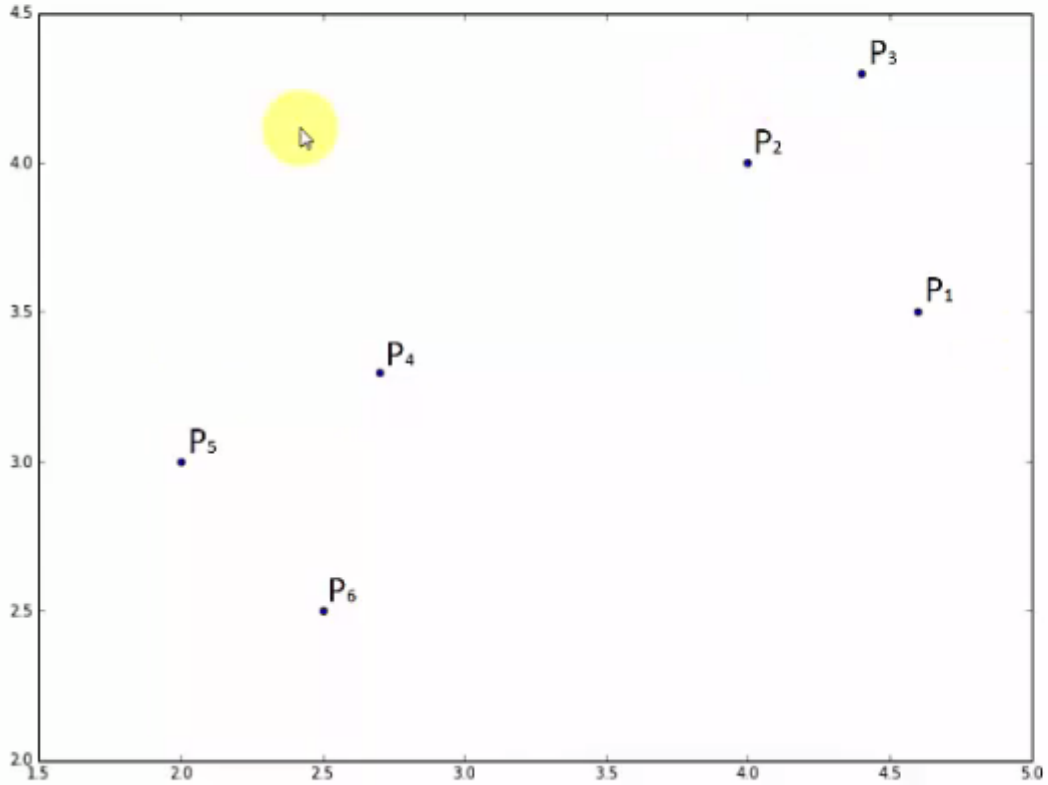


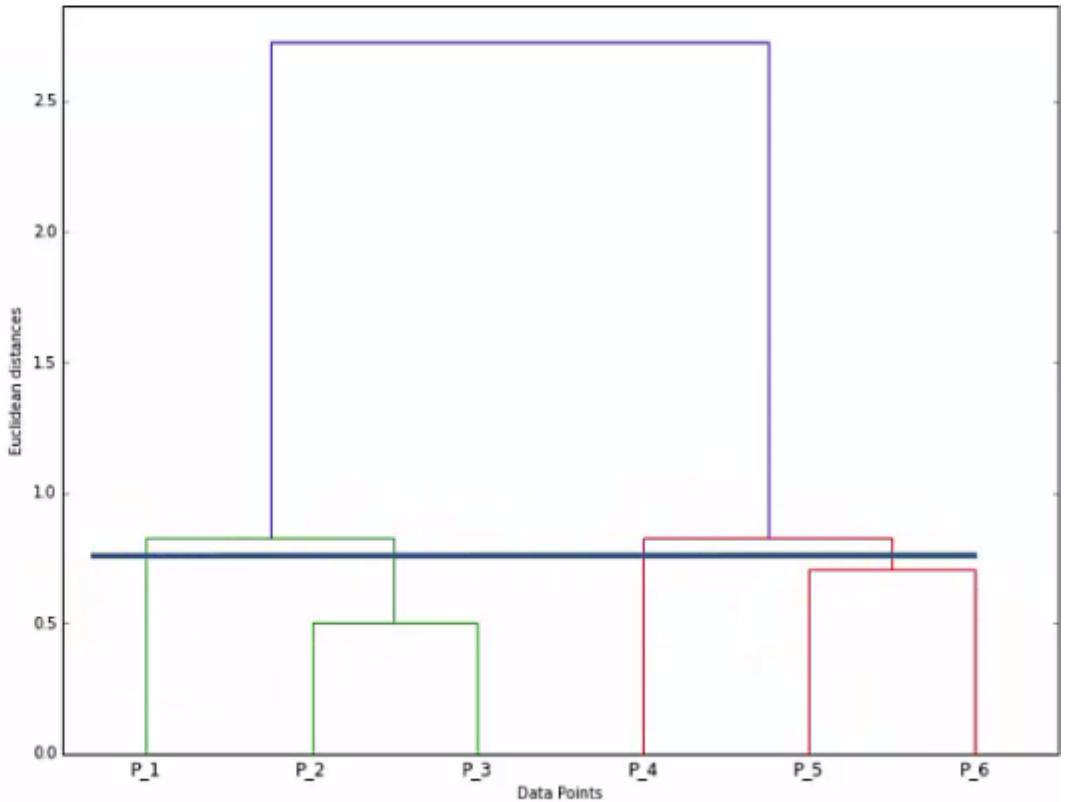
Dendrograms – Two Clusters



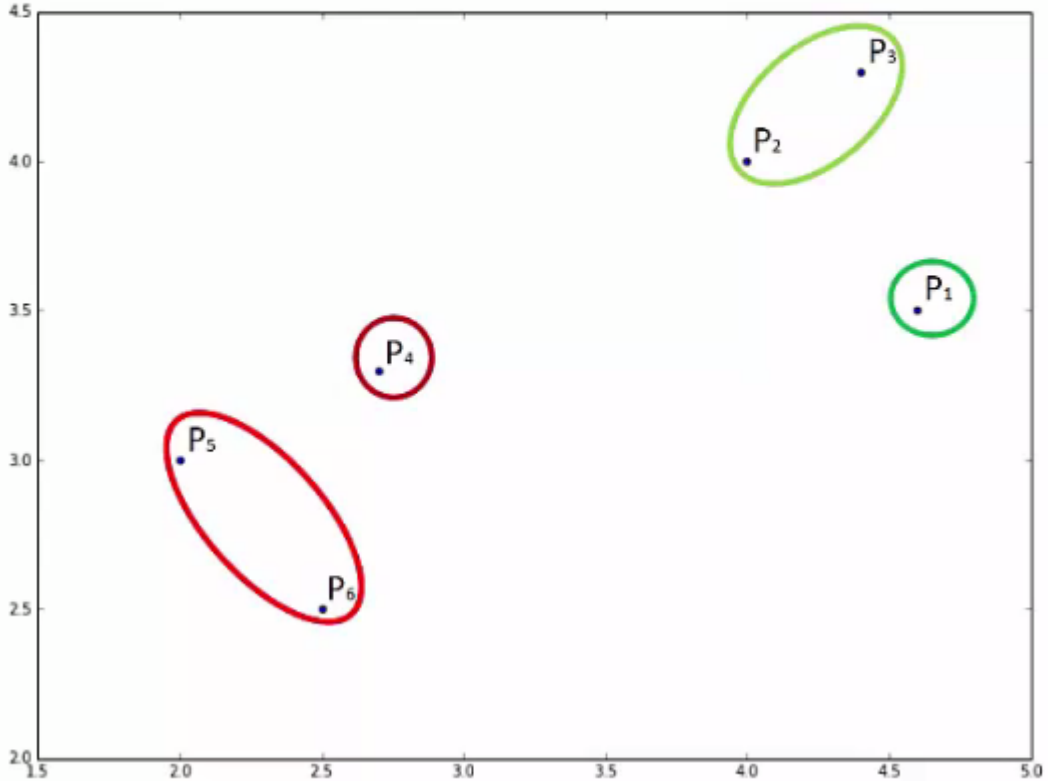


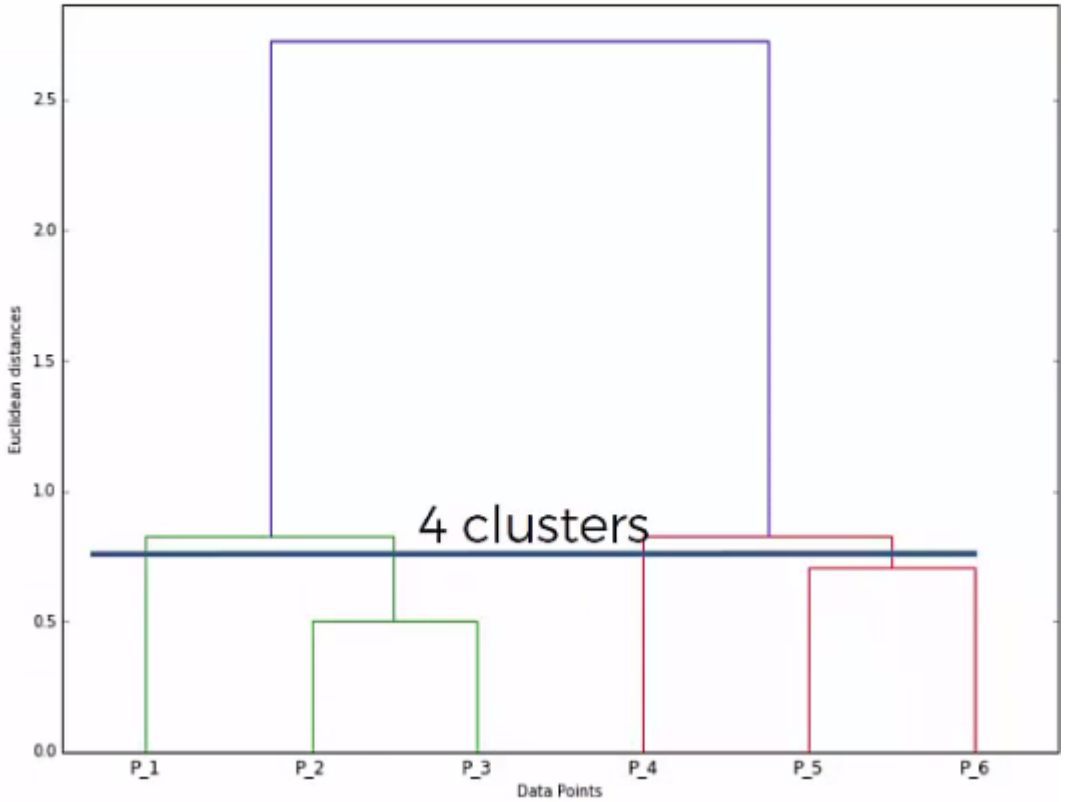
Dendrograms – Four Clusters



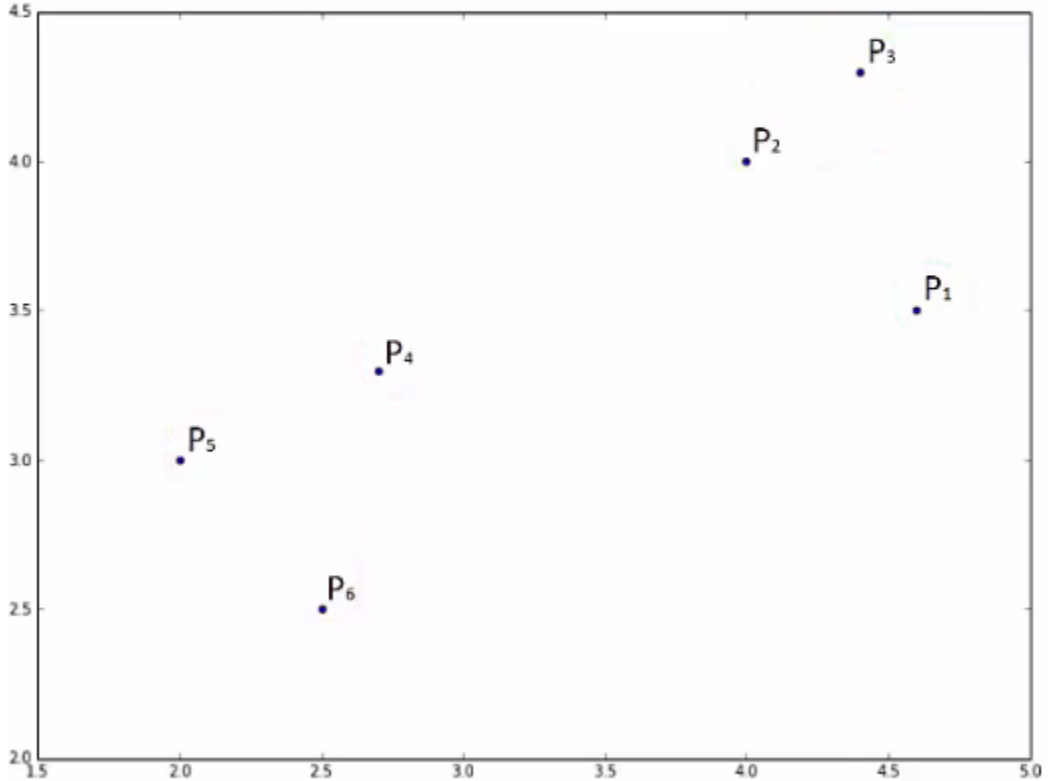


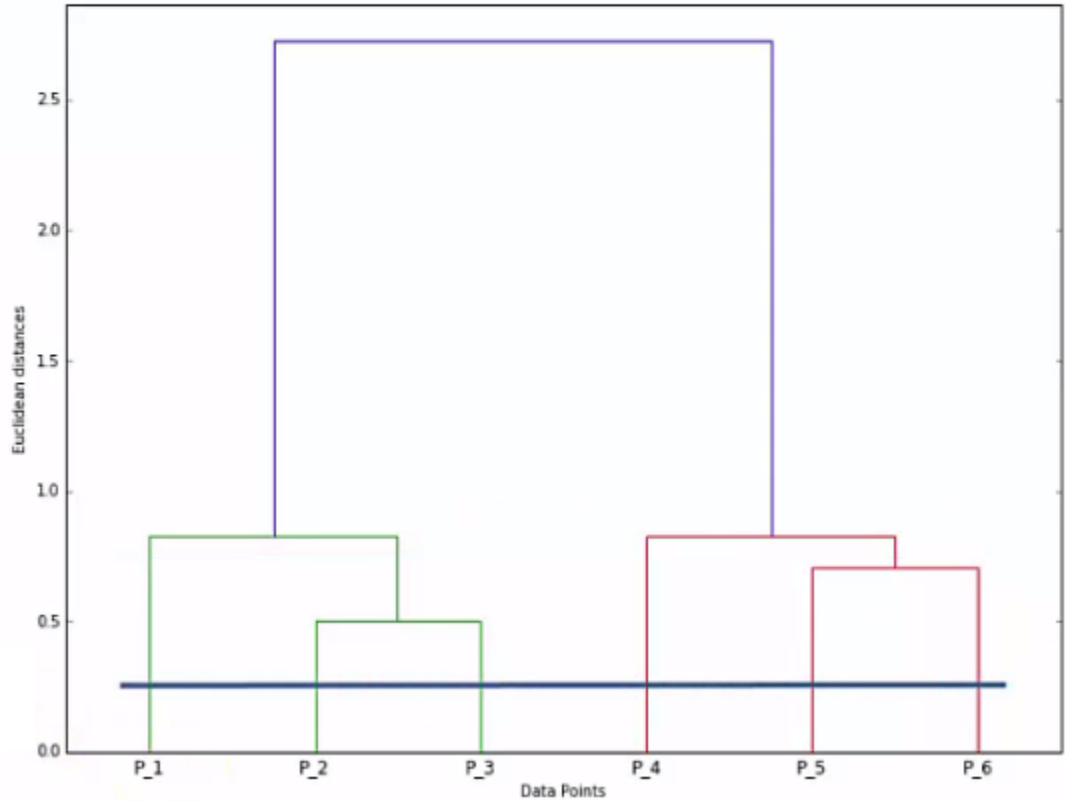
Dendrograms – Four Clusters



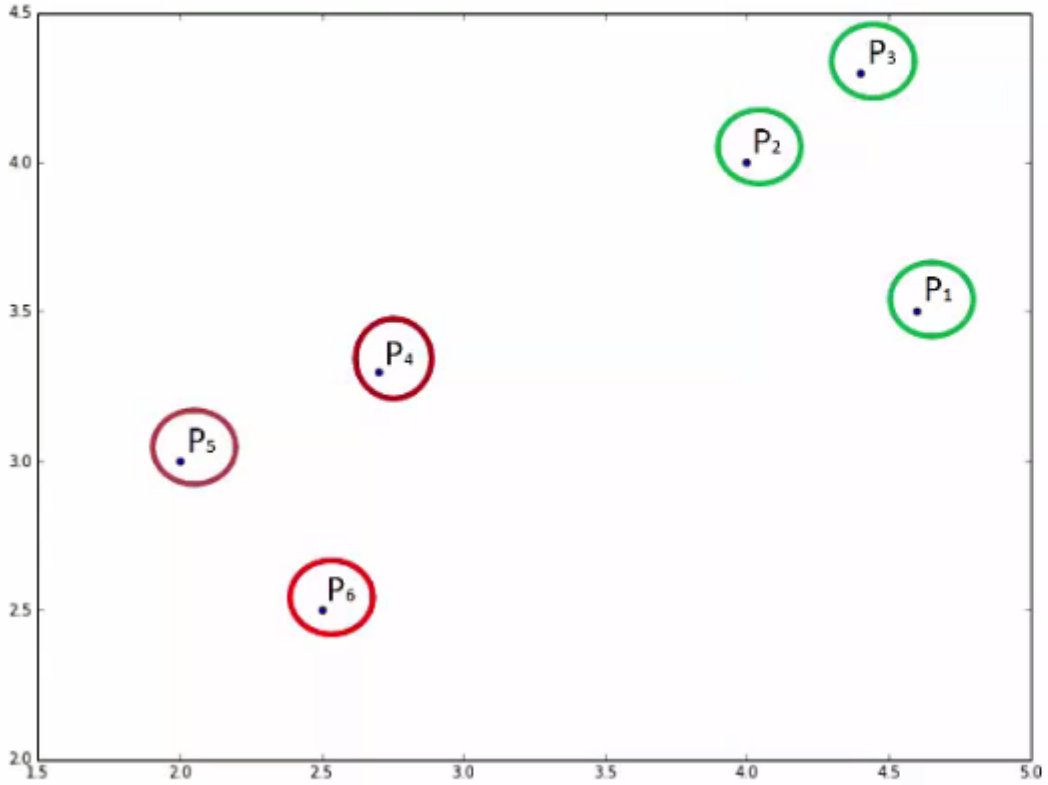


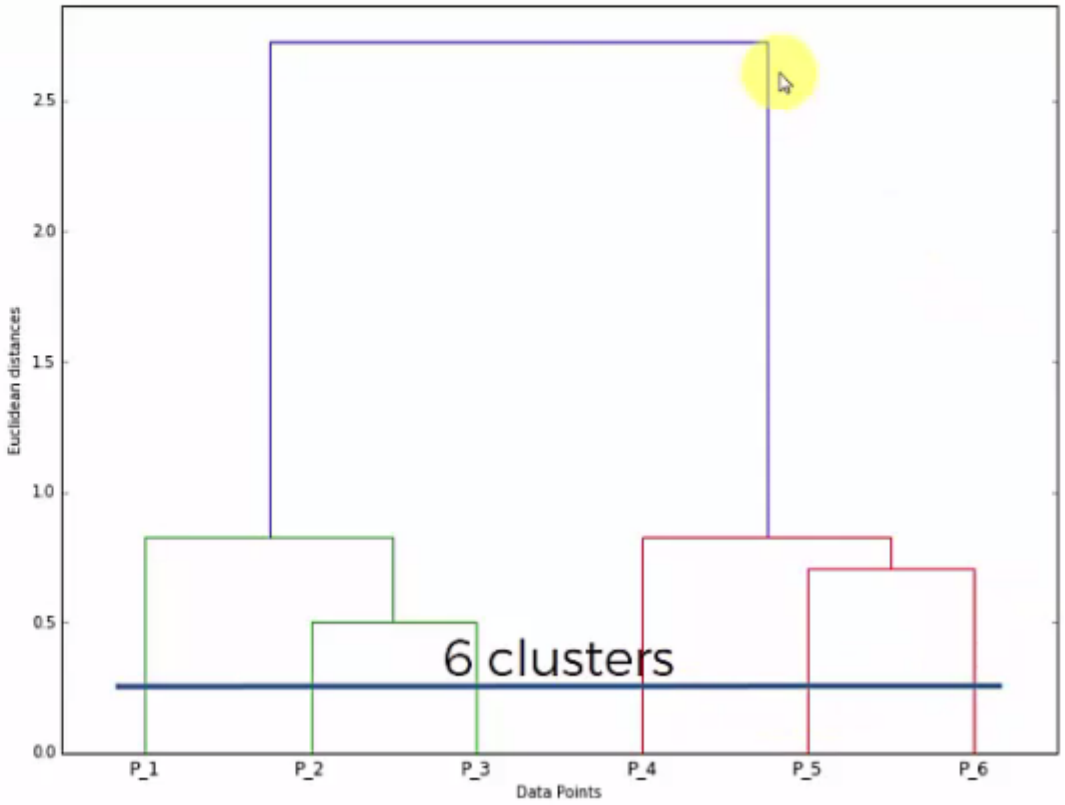
Dendrograms – Six Clusters



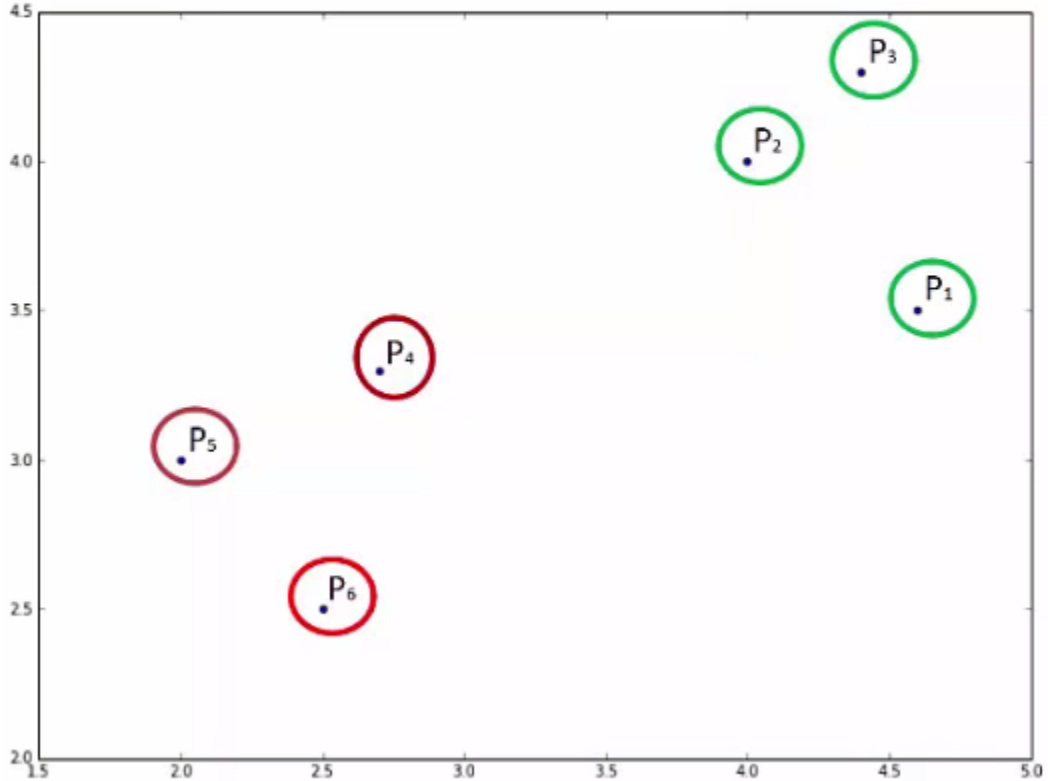


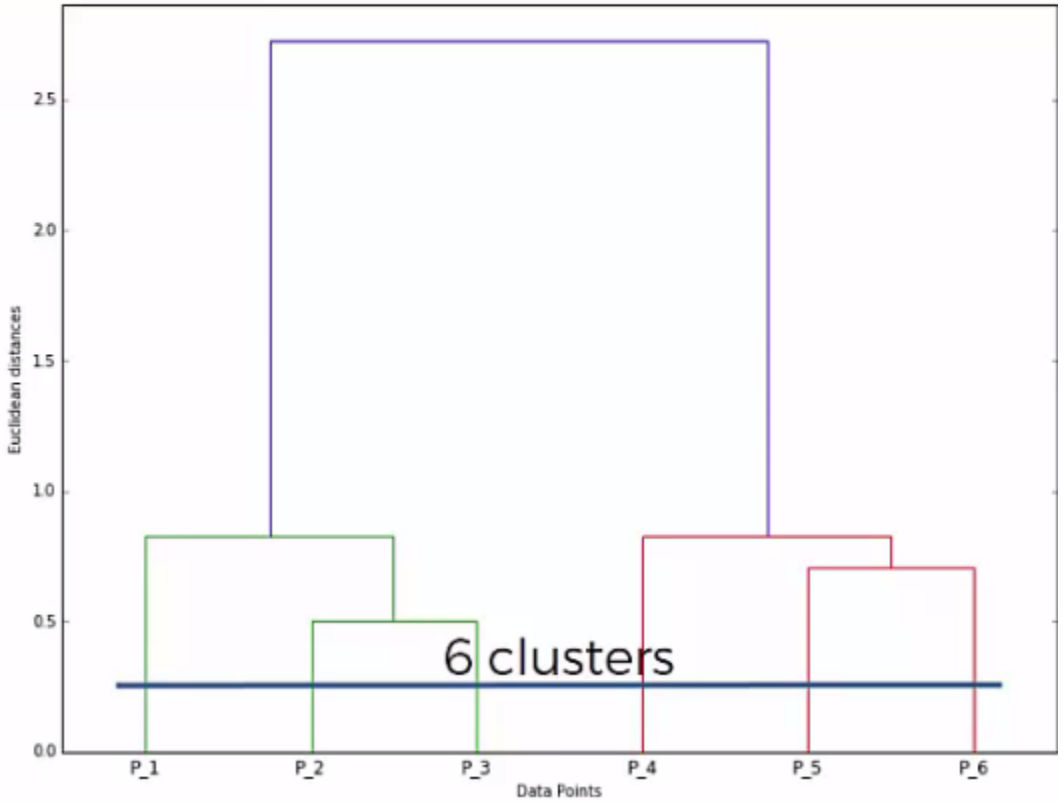
Dendrograms – Six Clusters



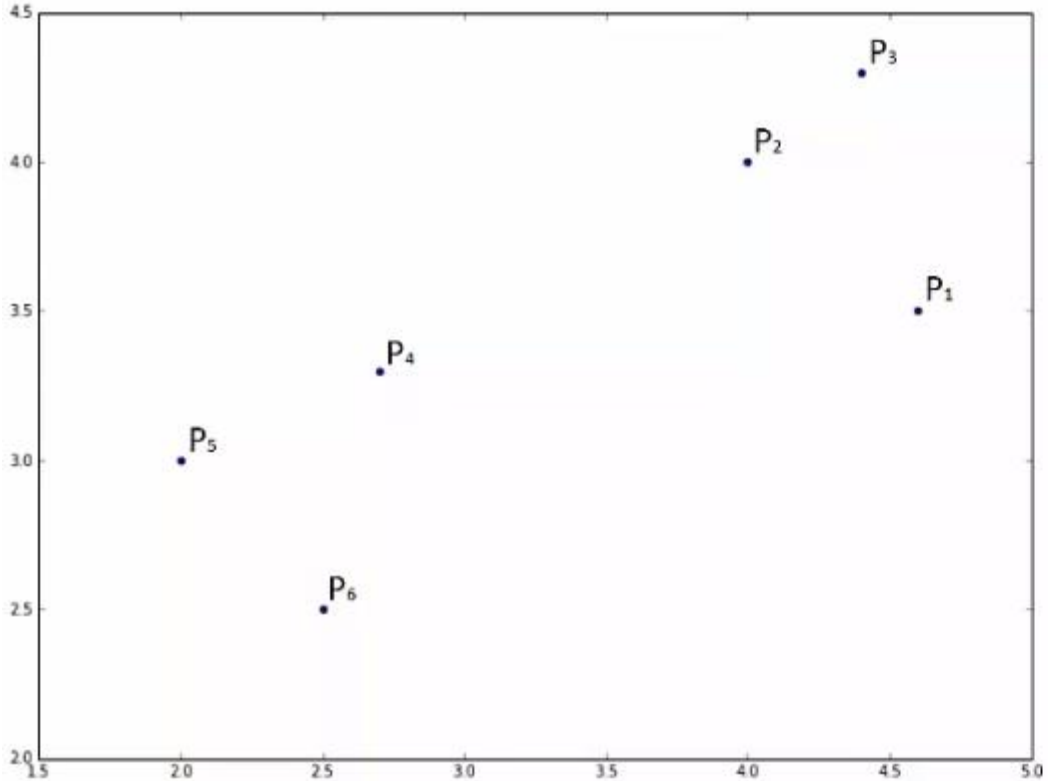


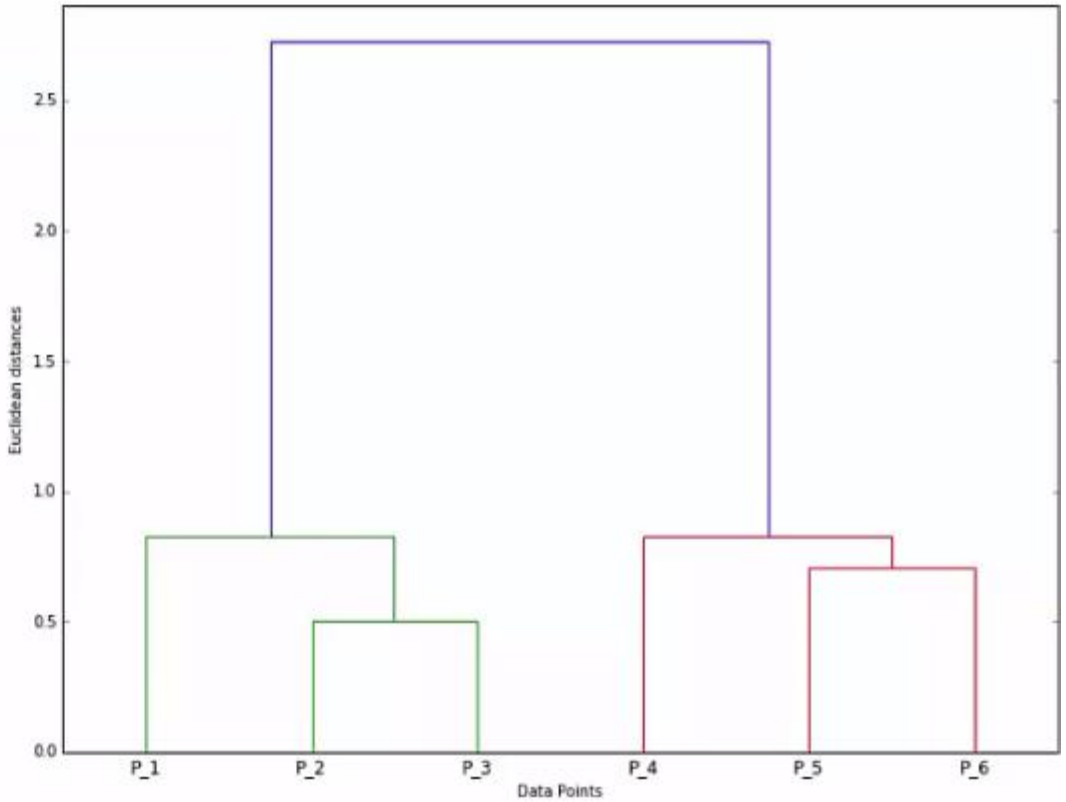
Dendrograms – Six Clusters



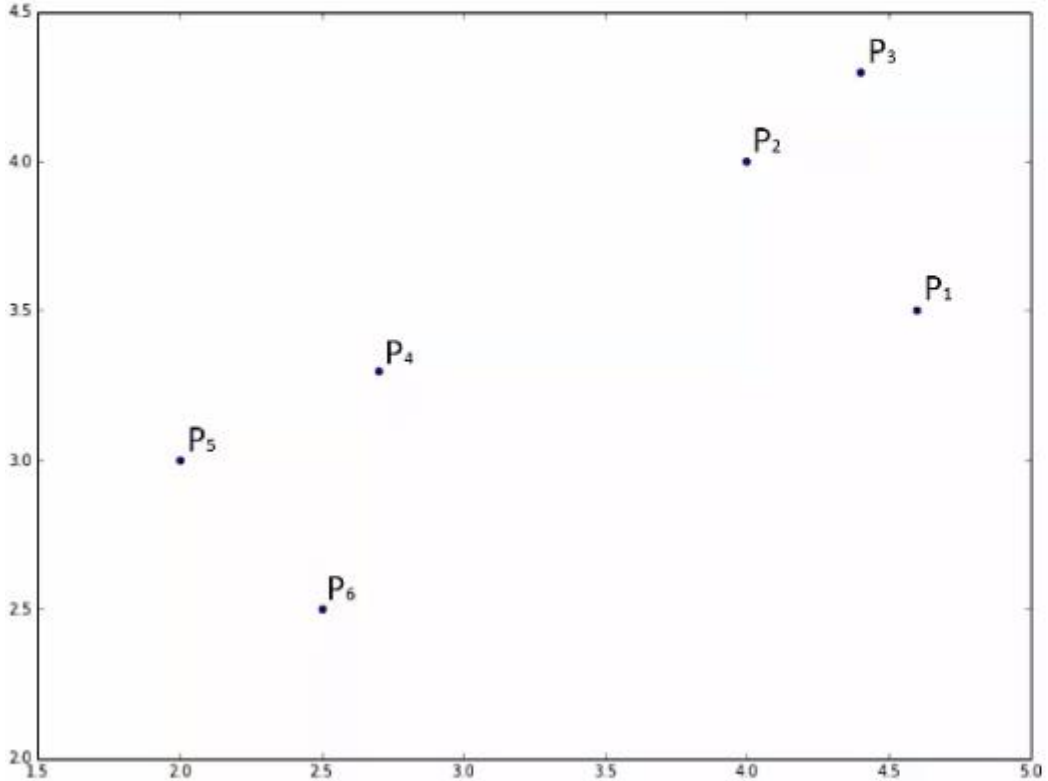


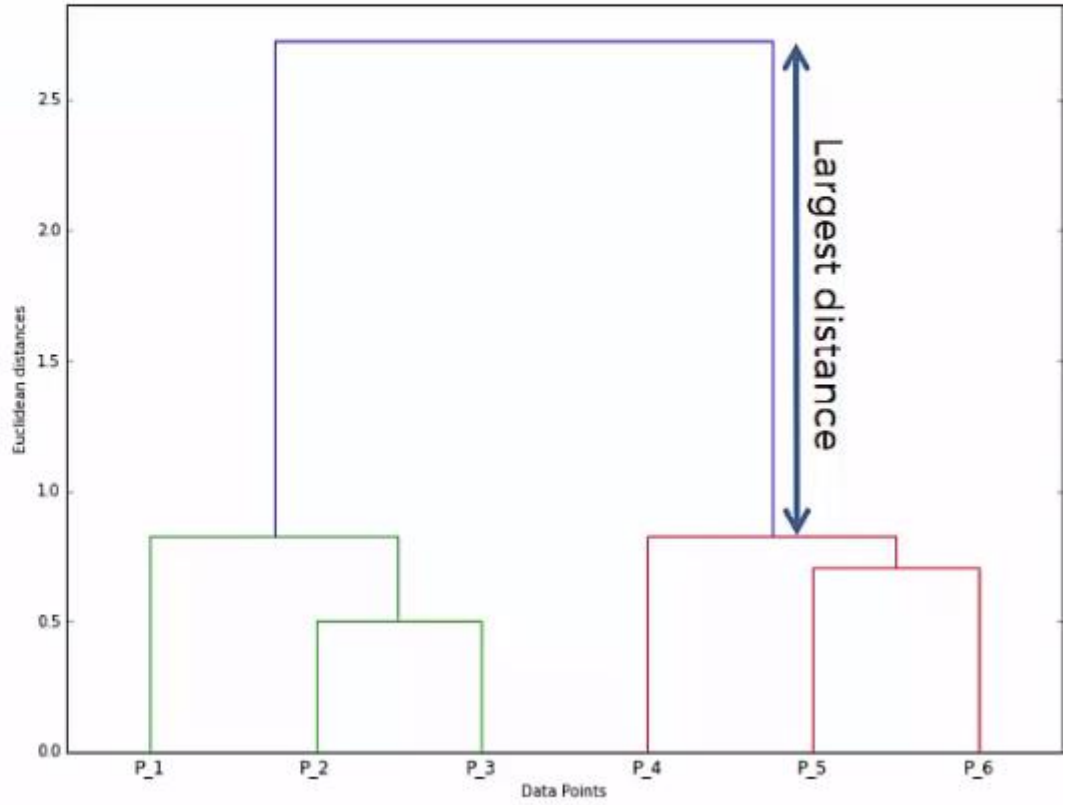
Dendrograms – Optimal # of Clusters



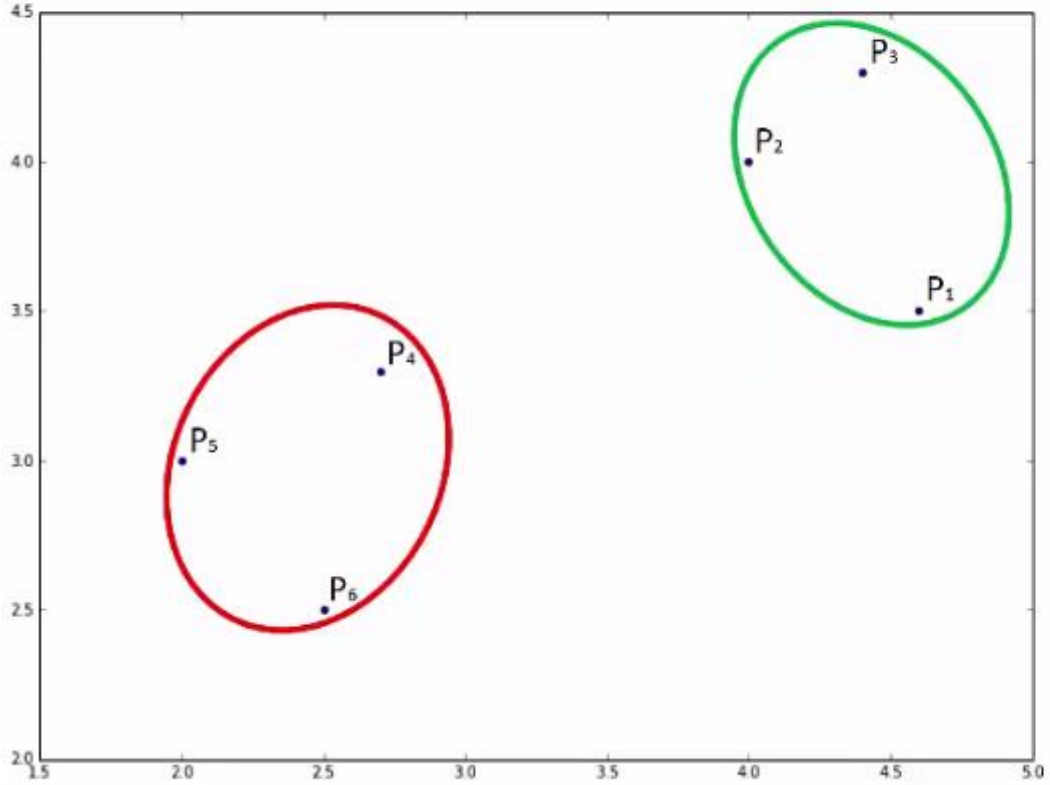


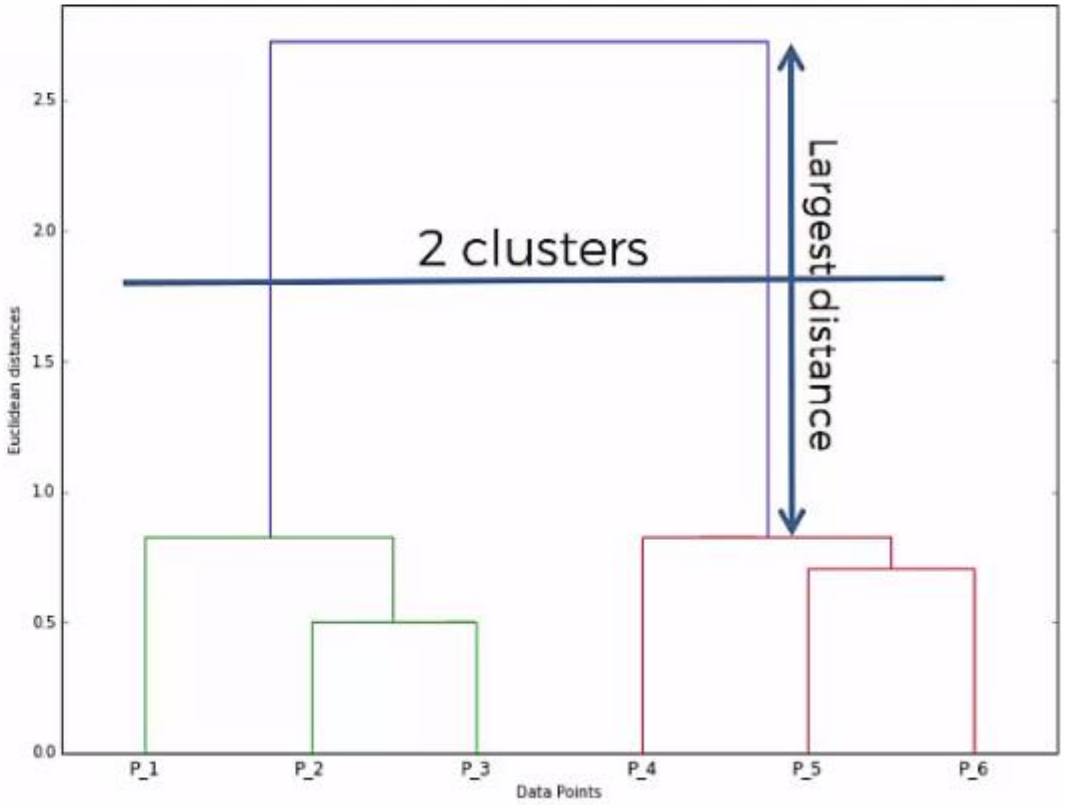
Dendrograms – Optimal # of Clusters





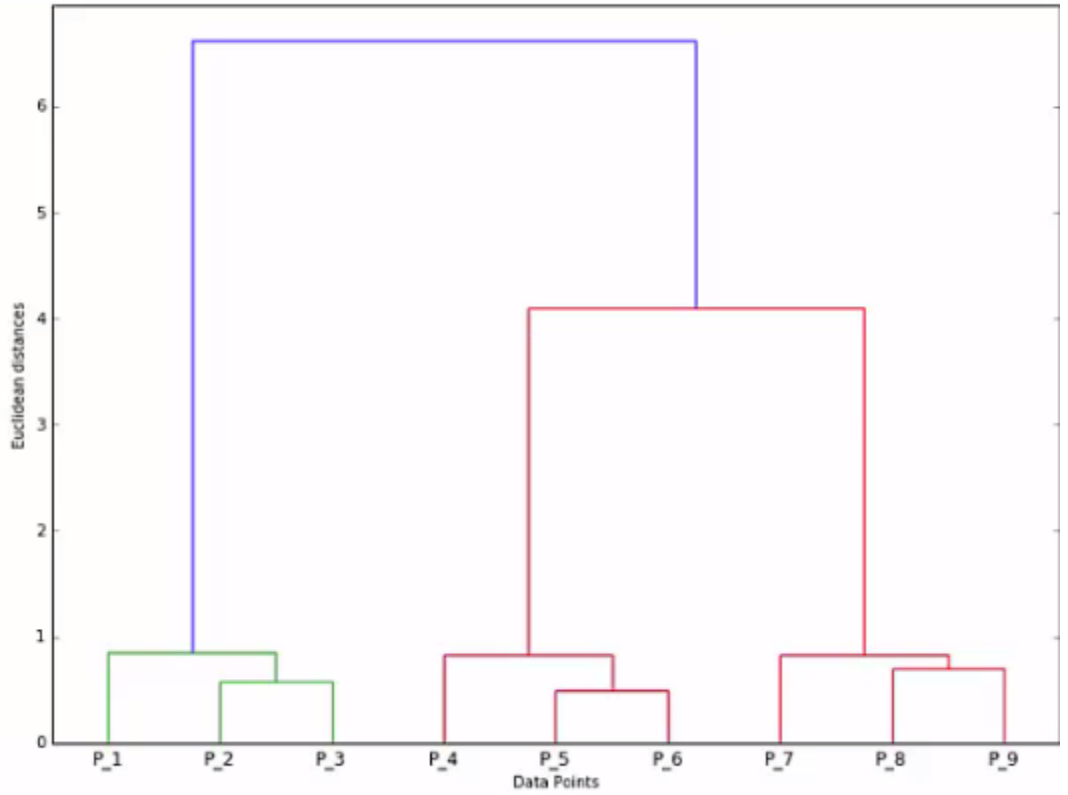
Dendrograms – Optimal # of Clusters



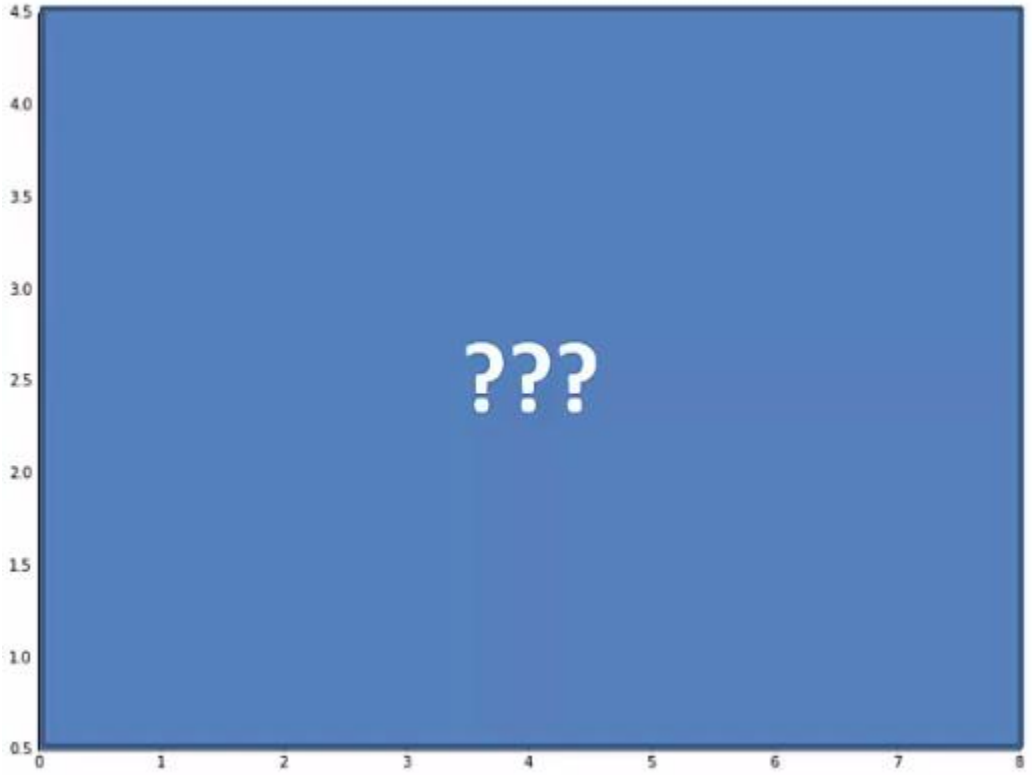


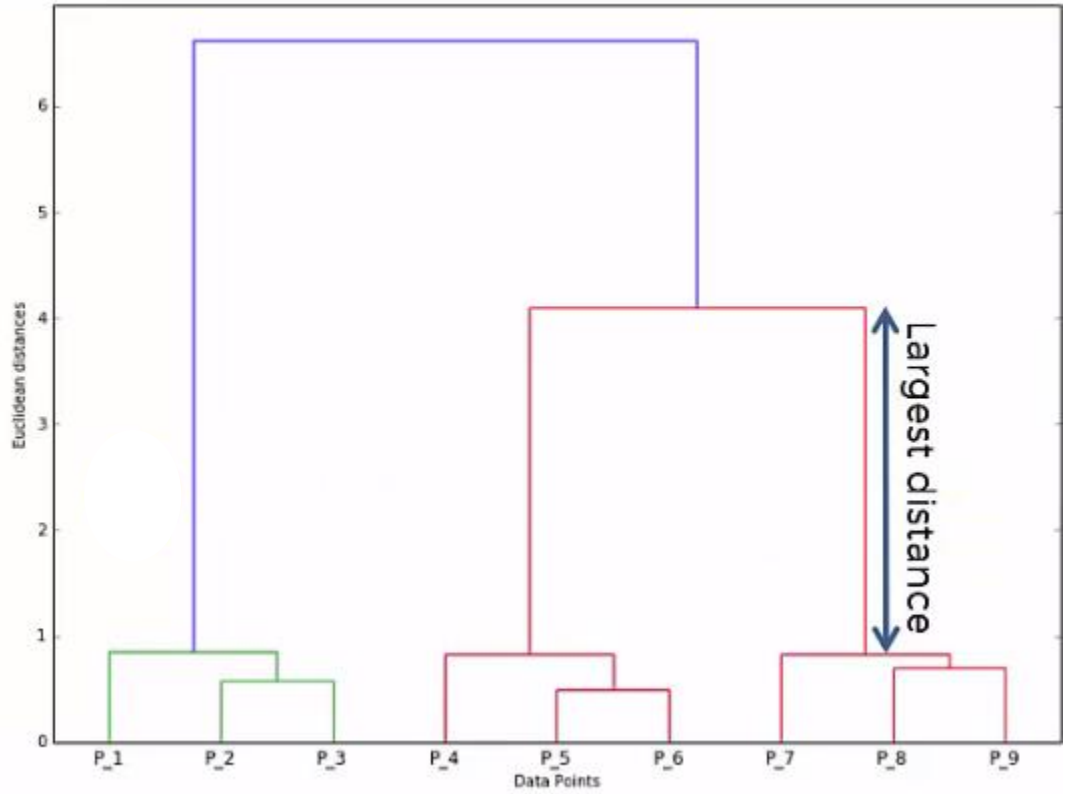
Dendrograms - How many clusters?



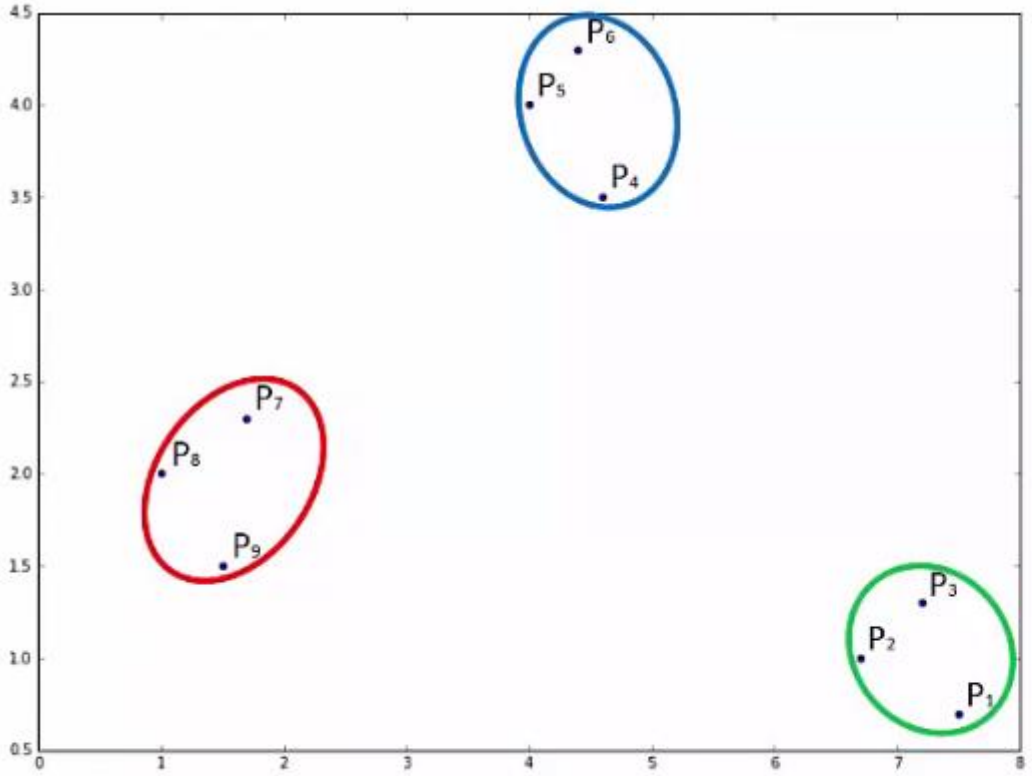


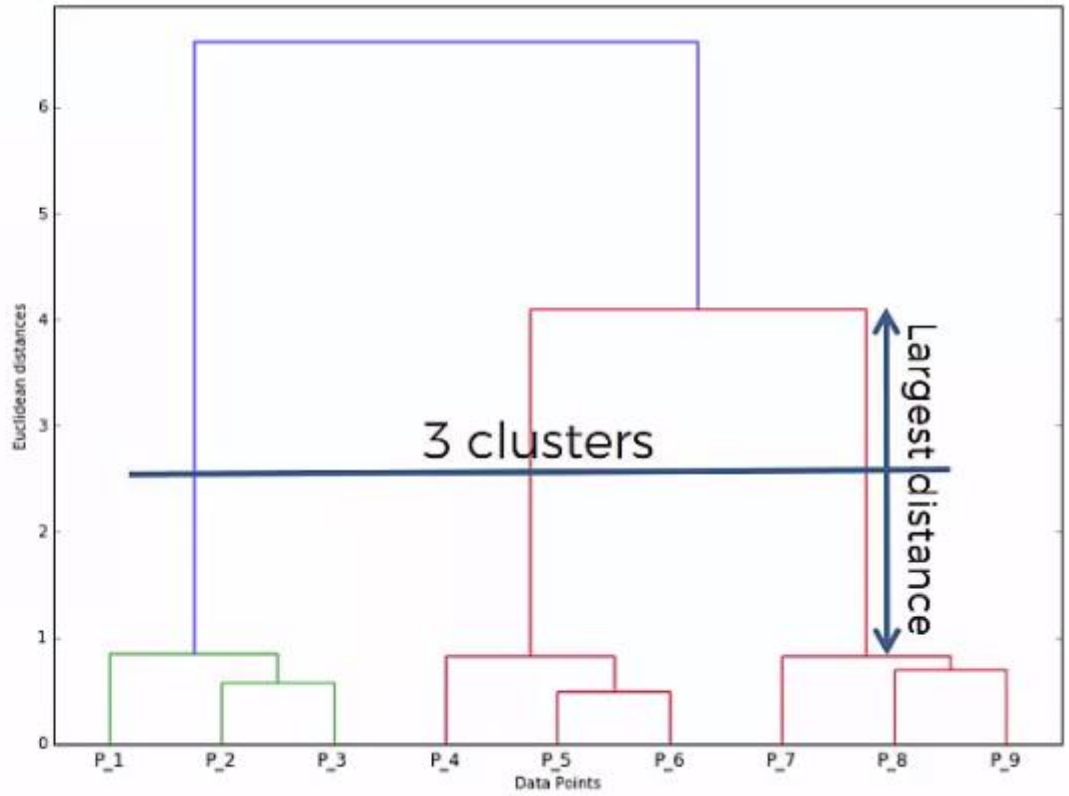
Dendrograms - How many clusters?





Dendrograms - How many clusters?





Single linkage example

Points	X	Y
A	1	2
B	2	3
C	3	1
D	5	4
E	6	5

```
#Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

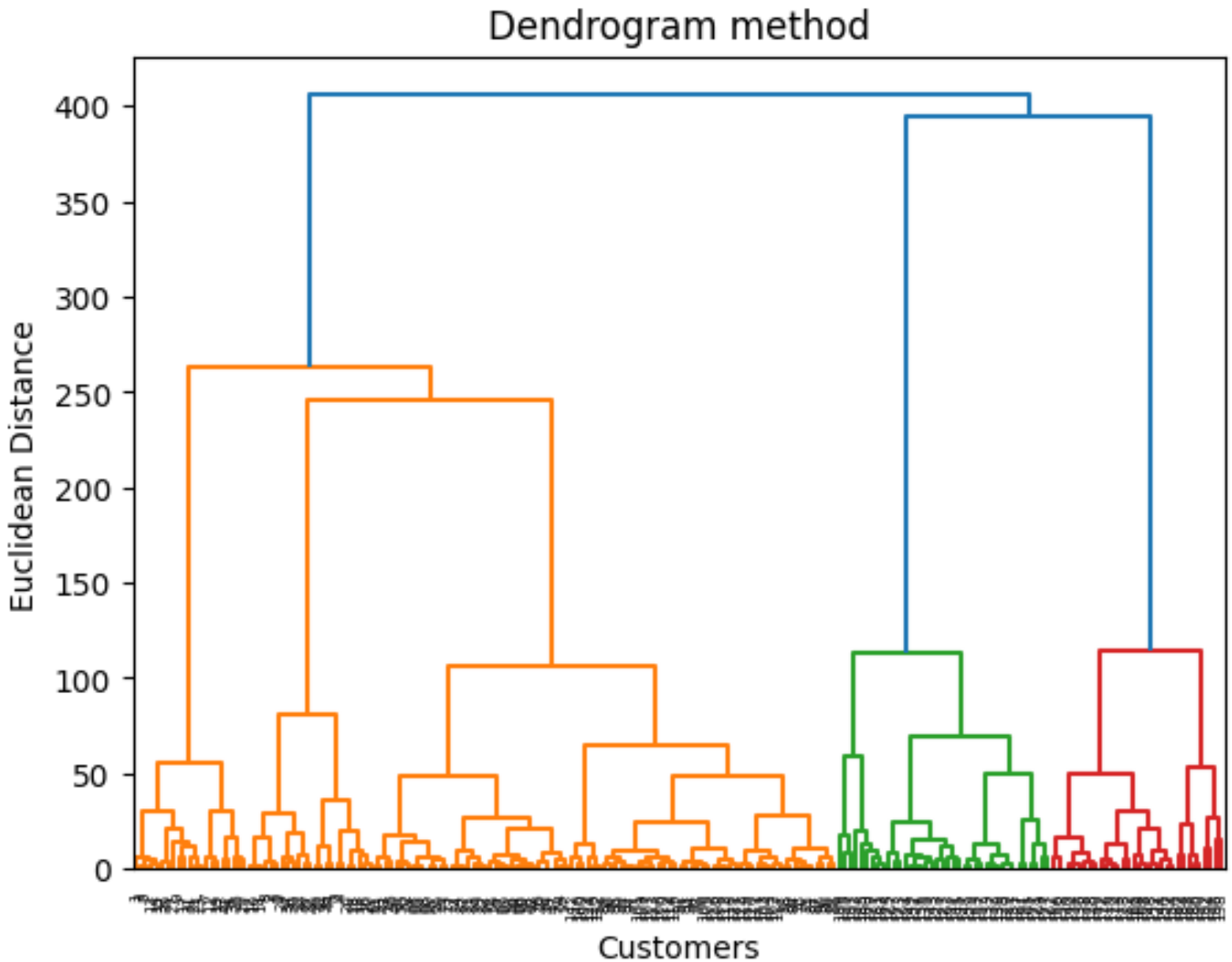
```
from google.colab import drive
drive.mount('/content/drive')
```

```
# Importing the mall dataset with pandas
import pandas as pd
data = pd.read_csv("drive/My Drive/Colab
Notebooks/DataSets/mall.csv")
dataset = data
```

```
X = dataset.iloc[:, [3, 4]]. values
```

```
#Using dendrogram method to find the optimal number of clusters
import scipy.cluster.hierarchy as sch
dendrogram = sch.dendrogram(sch.linkage (X, method ='ward'))

plt.title('Dendrogram method’)
plt.xlabel('Customers’)
plt.ylabel('Euclidean Distance’)
plt.show()
```

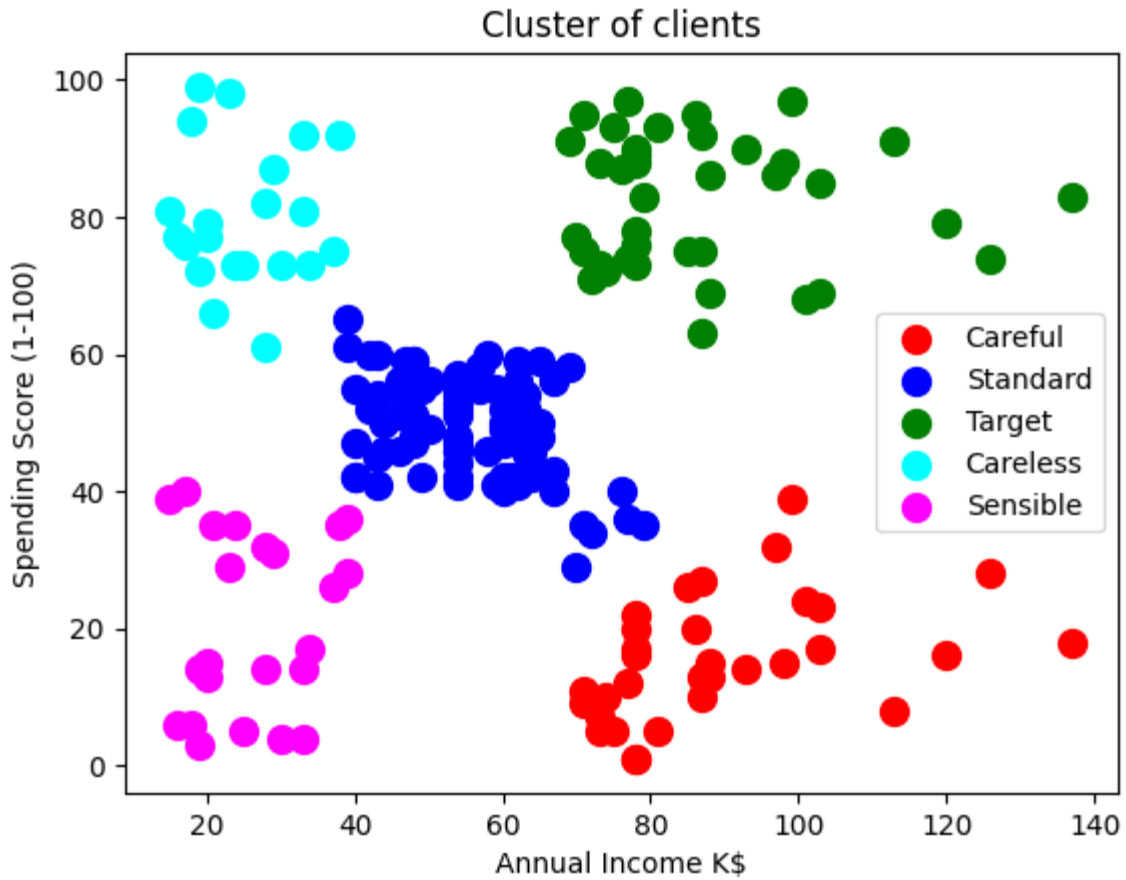

Dendrogram method
Euclidean Distance
400
350
300
250
200
150
100
50
0
Customers

```
#fitting HC to the mall dataset
#Agglomerative HC
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters =5, metric ='euclidean',
linkage='ward')

#each customer belongs to which cluster
y_hc = hc.fit_predict(X)
print(y_hc)
```

```
plt.scatter(X[y_hc==0, 0], X[y_hc==0, 1], s=100, c='red',
label = 'Careful')
plt.scatter(X[y_hc==1, 0], X[y_hc==1, 1], s=100, c='blue',
label = 'Standard')
plt.scatter(X[y_hc==2, 0], X[y_hc==2, 1], s=100, c='green',
label = 'Target')
plt.scatter(X[y_hc==3, 0], X[y_hc==3, 1], s=100, c='cyan',
label = 'Careless')
plt.scatter(X[y_hc==4, 0], X[y_hc==4, 1], s=100, c='magenta',
label = 'Sensible')
#plt.scatter(X[y_hc==5, 0], X[y_hc==5, 1], s=100, c='yellow',
label = 'New')

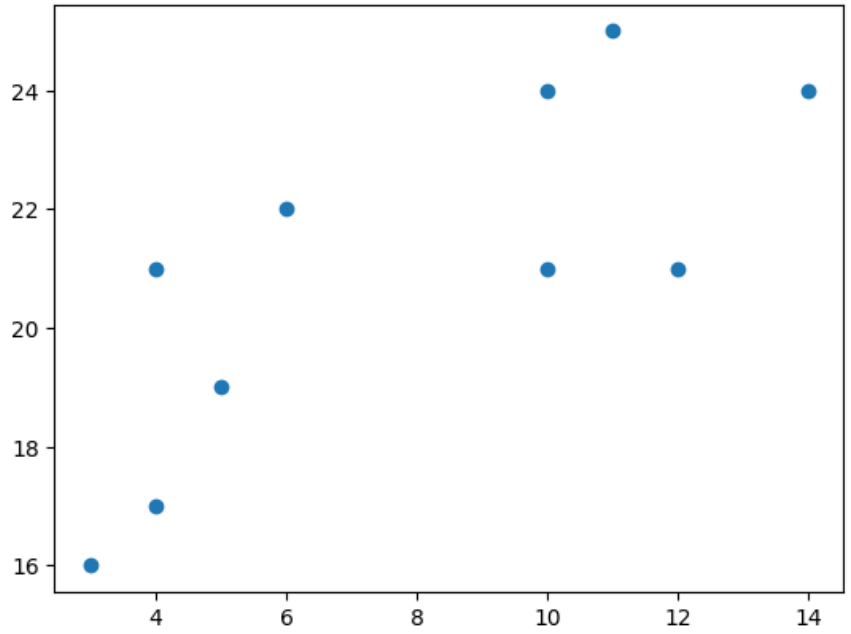
plt.title('Cluster of clients')
plt.xlabel('Annual Income K$')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```


Cluster of clients
Spending Score (1-100)
Annual Income K$
0
20
40
60
80
100
20
40
60
80
100
120
140
Careful
Standard
Target
Careless
Sensible

```
import numpy as np
import matplotlib.pyplot as plt

x = [4, 5, 10, 4, 3, 11, 14 , 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]

plt.scatter(x, y)
plt.show()
```



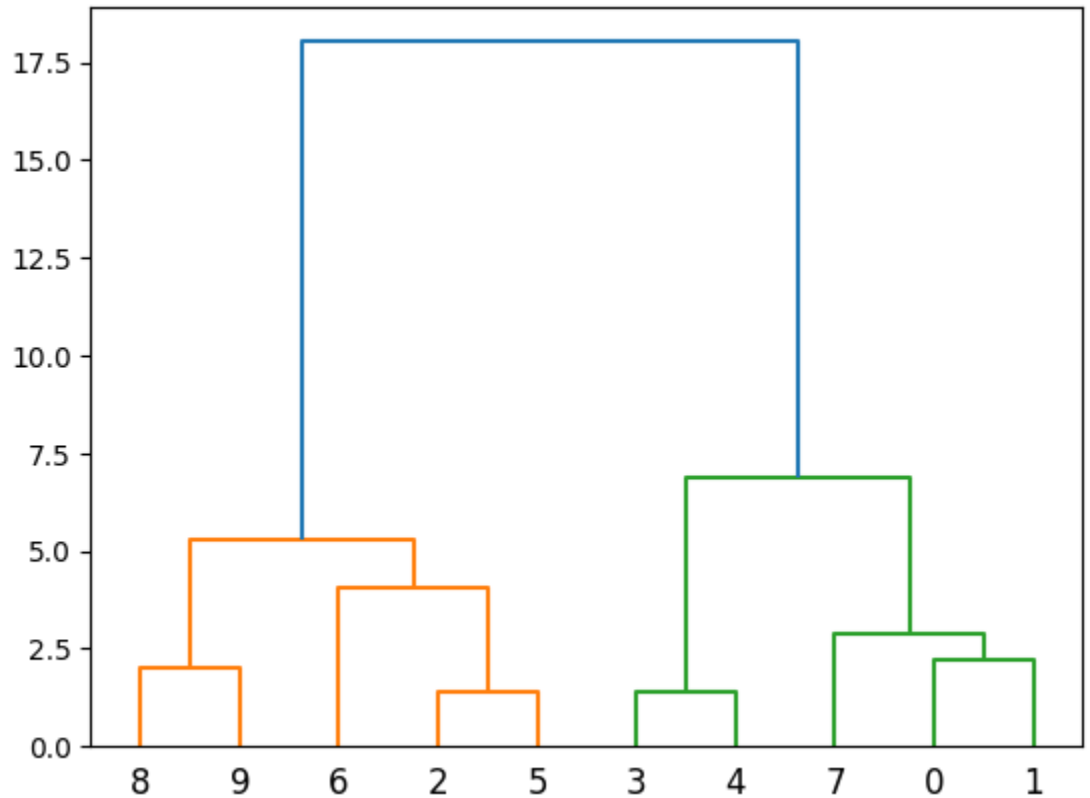
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

x = [4, 5, 10, 4, 3, 11, 14 , 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]

data = list(zip(x, y))

#‘ward’ minimizes the variance of the clusters being merged
linkage_data = linkage(data, method='ward', metric='euclidean’)
dendrogram(linkage_data)

plt.show()
```


17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
8
9
6
2
5
3
4
7
0
1

Thank you